

CRV

Docker

Massinissa Tighilt

Table of Contents

1 Introduction & Concepts Fondamentaux

2 Commandes Essentielles & Manipulation

3 Dockerfile & Construction d'Images

4 Réseaux Docker

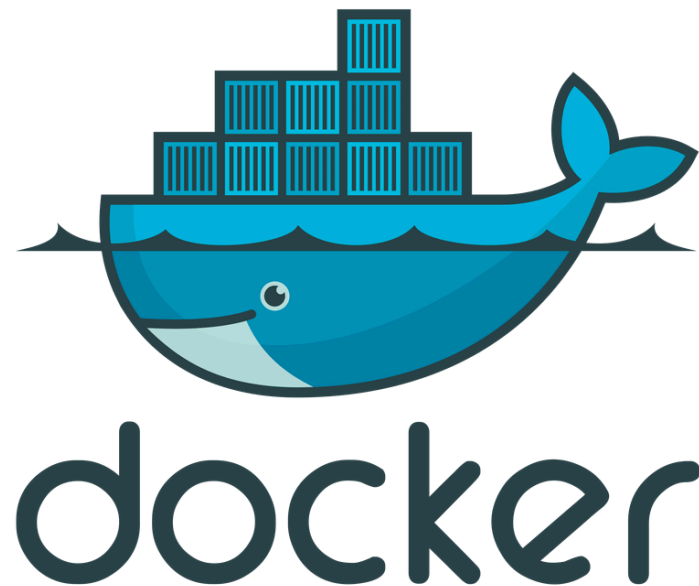
5 Volumes & Persistance

6 Docker Compose

7 Bonnes pratiques & Pièges
Fréquents

Introduction & Concepts Fondamentaux

- Objectif : être autonome et prêt pour les certification type "The Docker Certified Associate (DCA)"
- Chaque partie : notions → commandes → mini-démo → pièges fréquents



Introduction & Concepts Fondamentaux

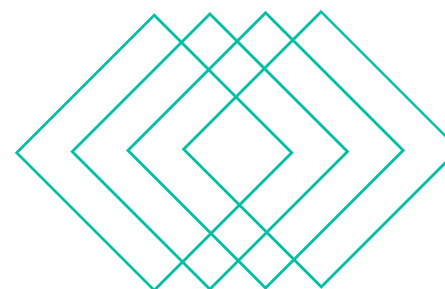
Pourquoi Docker ?

- Reproductibilité : même environnement partout (PC, VM, CI, prod)
- Isolation : dépendances, ports, processus, système de fichiers
- Vitesse : démarrage en secondes, images partageables
- Standard de fait en DevOps / Cloud

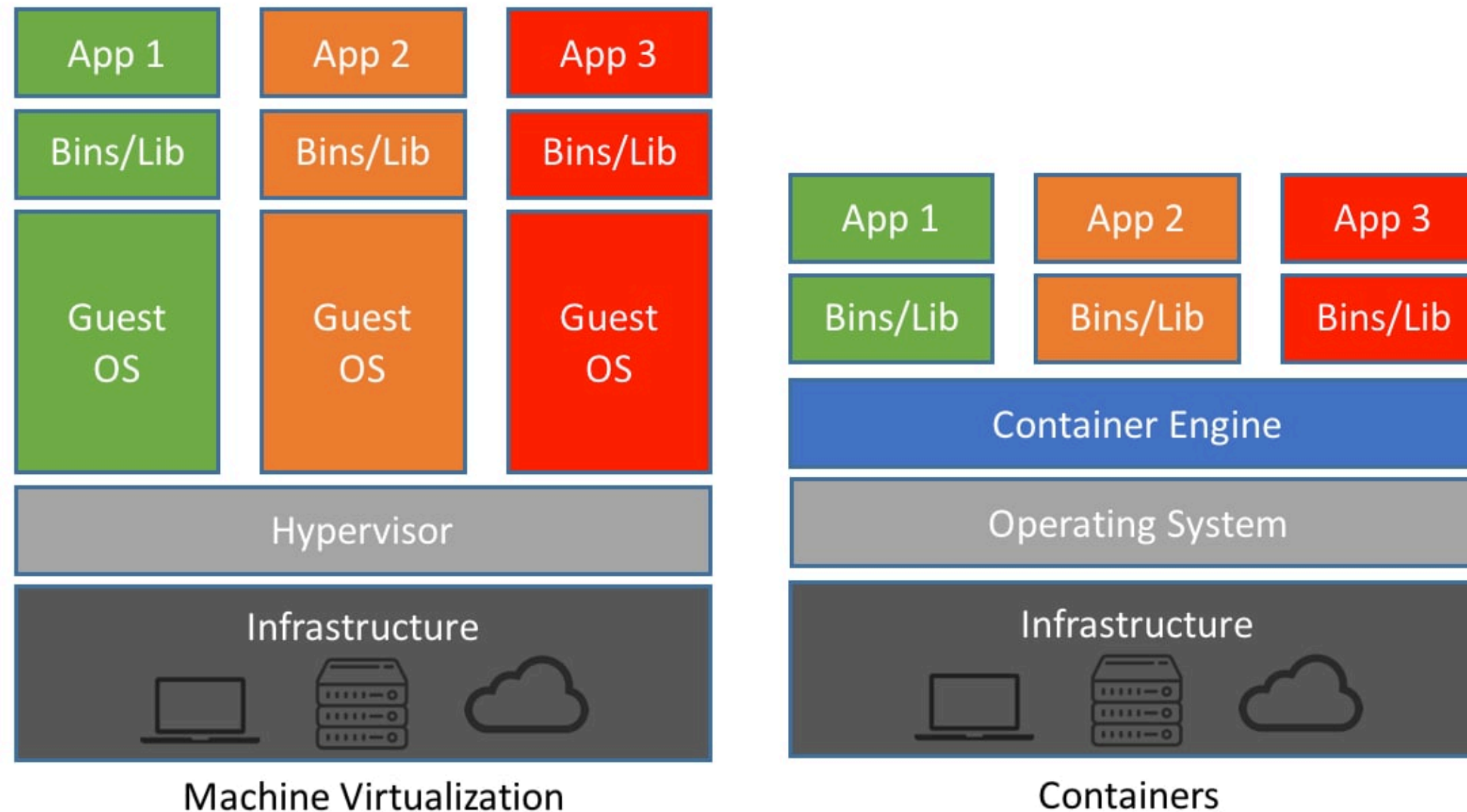


Problème classique :

- “Ça marche sur ma machine”
- versions / OS / libs / config différentes



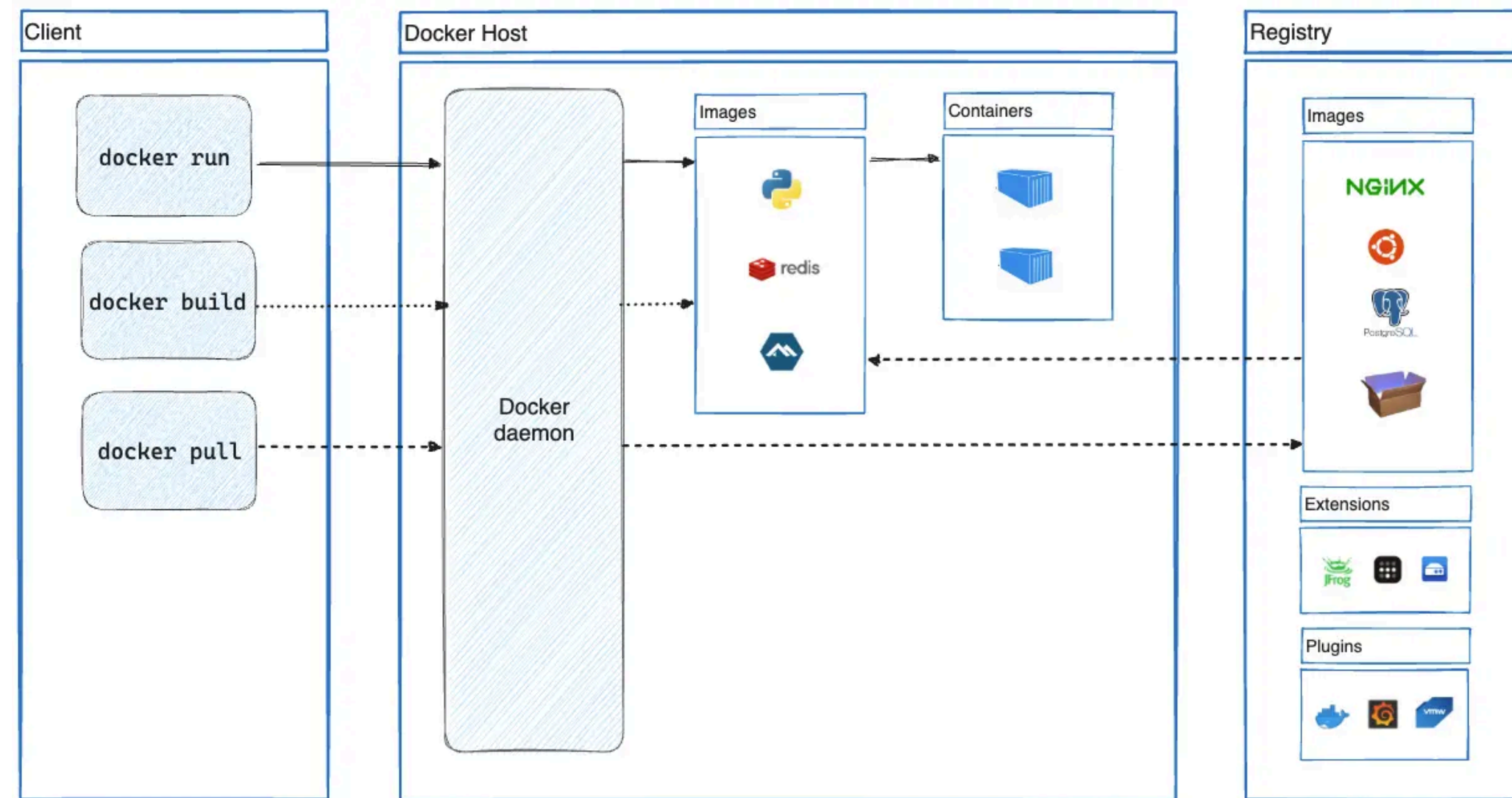
Introduction & Concepts Fondamentaux



- VM : OS complet + hyperviseur → plus lourd
- Docker : partage le kernel Linux → plus léger
- Conteneurs = namespaces + cgroups + filesystem en couches
- Sur Mac/Windows : Docker Desktop utilise une VM légère

Concepts Fondamentaux

Architecture Docker



- Architecture client-serveur
- Le daemon exécute les opérations lourdes (construction d'images, exécution de conteneurs, gestion des réseaux et volumes)
- Communication via API REST (Socket UNIX (local), Interface réseau (remote))
- Le client et le daemon peuvent être :
 - sur la même machine
 - sur des machines différentes

Concepts Fondamentaux

Architecture Docker

Communication Client ↔ Daemon

```
docker run nginx
↓
Requête API REST
↓
Daemon (dockerd) exécute la
commande
```

- Le client utilise l'API Docker
- Plusieurs clients peuvent parler à un même daemon
- Communication possible avec un daemon distant

Concepts Fondamentaux

Architecture Docker

Communication Client ↔ Daemon

```
graph TD; A[docker run nginx] --> B[Requête API REST]; B --> C[Daemon (dockerd) exécute la commande];
```

- Le client utilise l'API Docker
- Plusieurs clients peuvent parler à un même daemon
- Communication possible avec un daemon distant

Le Docker Daemon (dockerd)

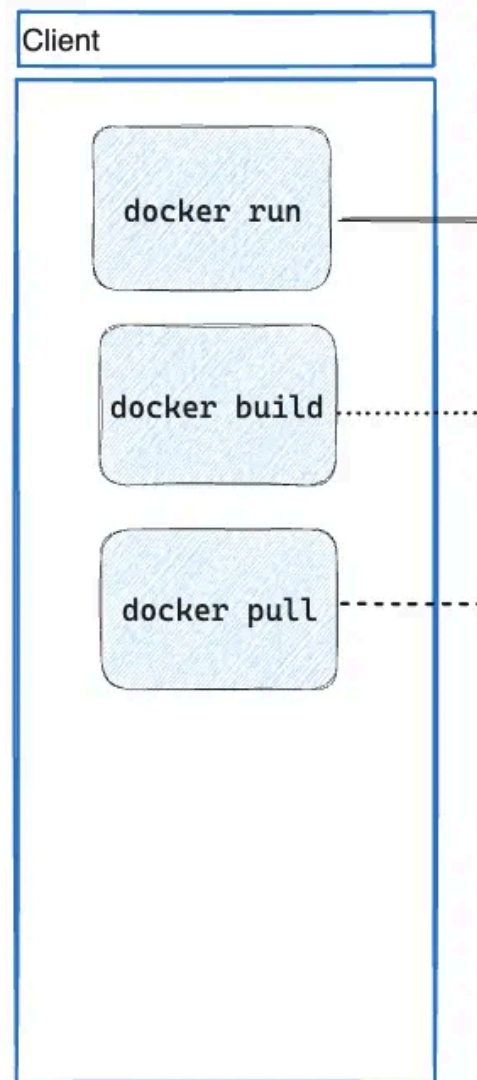
- Écoute les requêtes API Docker
- Gère les objets Docker :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
- Peut communiquer avec d'autres daemons
- Exécute réellement les conteneurs

C'est le "moteur" de Docker.

Concepts Fondamentaux

Architecture Docker

Le Docker Client (docker)

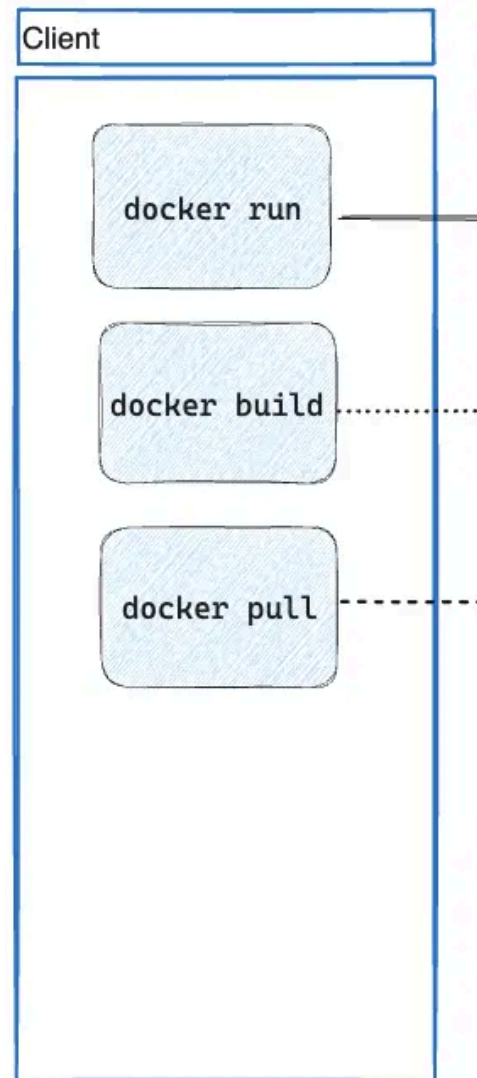


- Le client Docker est l'interface principale des utilisateurs.
 - docker run
 - docker build
 - docker pull
 - docker push
 - ...
- Envoie les commandes au daemon
- Utilise l'API Docker
- Peut se connecter à plusieurs daemons
- C'est l'outil en ligne de commande.

Concepts Fondamentaux

Architecture Docker

Docker Compose (Client avancé)

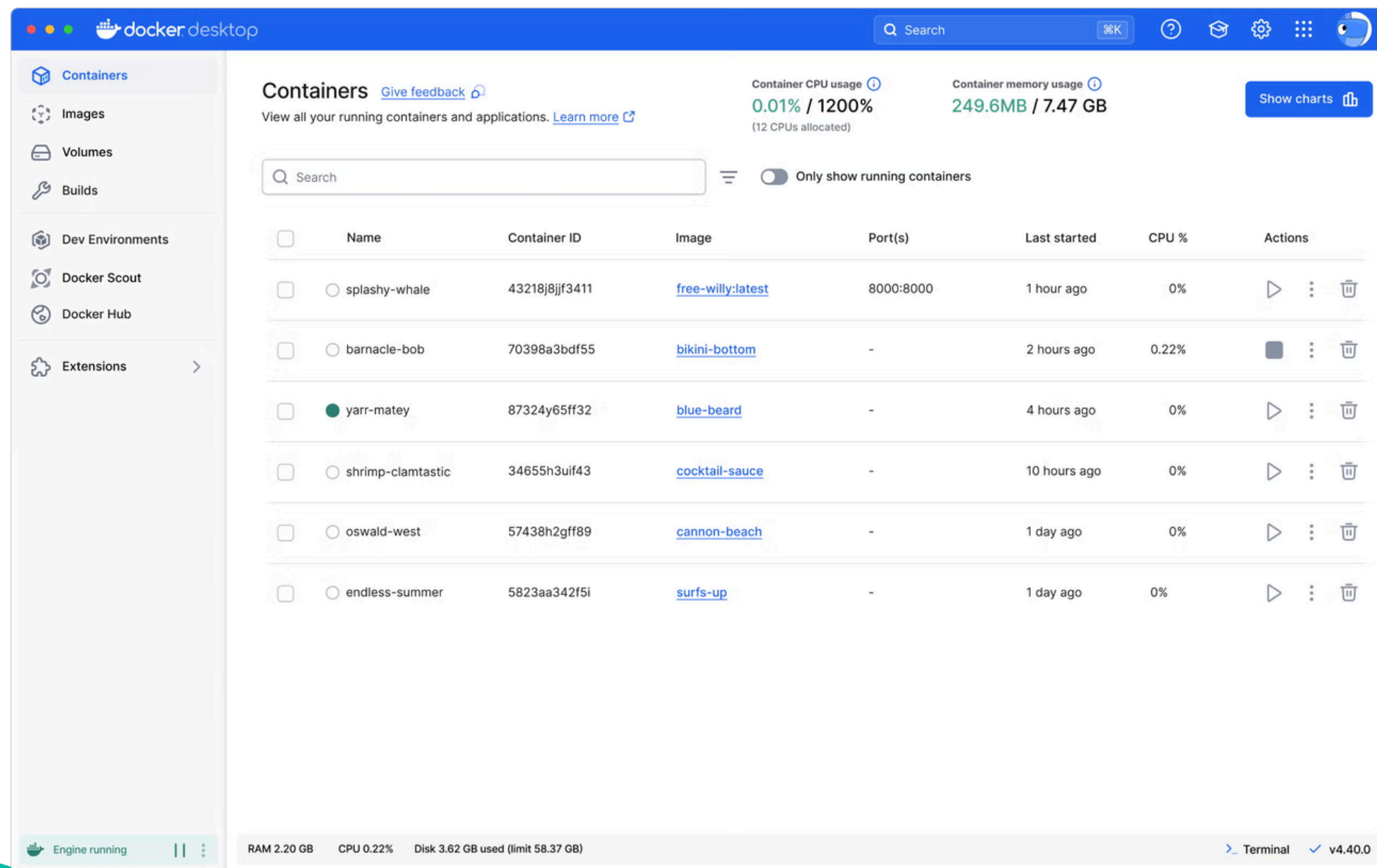


- Docker Compose est un autre client Docker.
- Il permet de :
 - Gérer des applications multi-conteneurs
 - Définir l'infrastructure dans un fichier YAML
 - Lancer toute l'application avec une seule commande
 - `docker compose up`
- Idéal pour projets web (front + back + base de données).

Concepts Fondamentaux

Architecture Docker

Docker Desktop

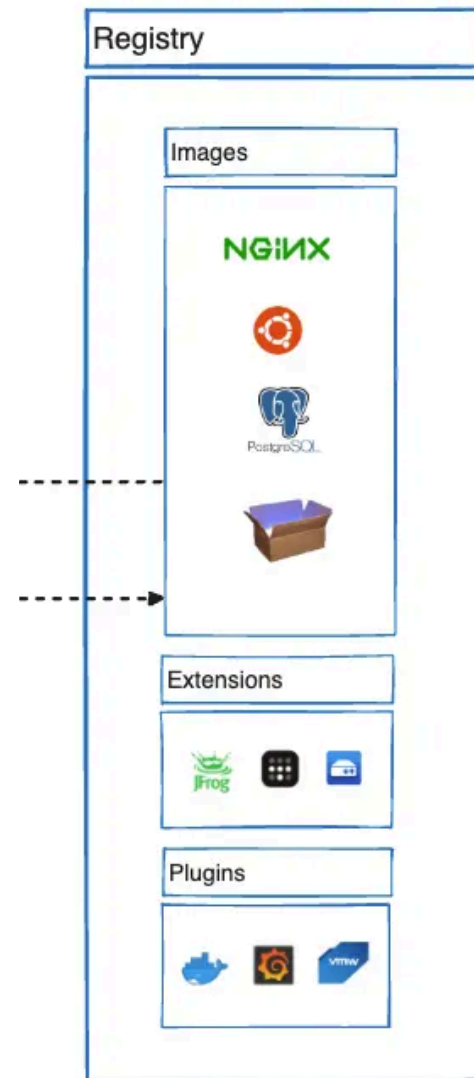


- Docker Desktop est une application pour :
 - macOS
 - Windows
 - Linux
- Elle inclut :
 - Docker daemon
 - Docker client
 - Docker Compose
 - Kubernetes
- Gestion des credentials
- Sur Mac/Windows, Docker fonctionne dans une VM légère.

Concepts Fondamentaux

Architecture Docker

Les Registres Docker (Docker Registry)



- Un registre stocke des images Docker.
- Exemples :
 - Docker Hub (public)
 - Registres privés
 - Registry auto-hébergé
- Par défaut, Docker utilise Docker Hub.
- Quand tu exécutes :
 - **docker pull nginx**
- Docker :
 - Télécharge l'image depuis le registry configuré
- Quand tu exécutes :
 - **docker push monimage**
- Docker :
 - Envoie l'image vers le registry
- Permet le partage et le déploiement.

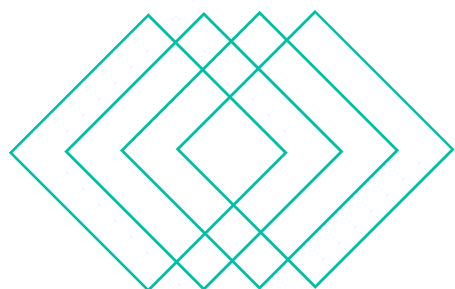


Concepts Fondamentaux

Objets Docker



- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins





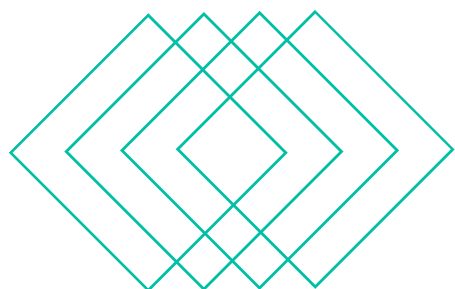
Concepts Fondamentaux

Objets Docker

- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins

Les Images Docker

- Une image est :
 - Un modèle en lecture seule
 - Une suite d'instructions
 - Une base pour créer des conteneurs
- Exemple :
 - Image Ubuntu
 - Image Python
 - Image Nginx





Concepts Fondamentaux

Objets Docker



- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins

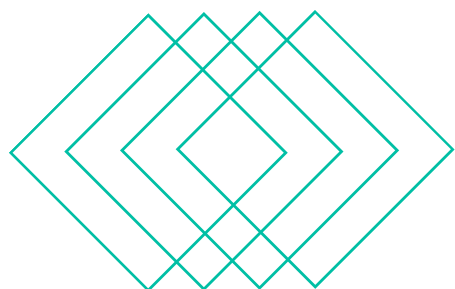
Structure d'une Image

- Une image est composée de :
 - Plusieurs couches (layers)
 - Chaque instruction Dockerfile crée une couche
- Exemple :
 - **FROM ubuntu**
 - **RUN apt install python**
 - **COPY app.py**

Chaque ligne crée une nouvelle couche.

Les Images Docker

- Une image est :
 - Un modèle en lecture seule
 - Une suite d'instructions
 - Une base pour créer des conteneurs
- Exemple :
 - Image Ubuntu
 - Image Python
 - Image Nginx



Concepts Fondamentaux

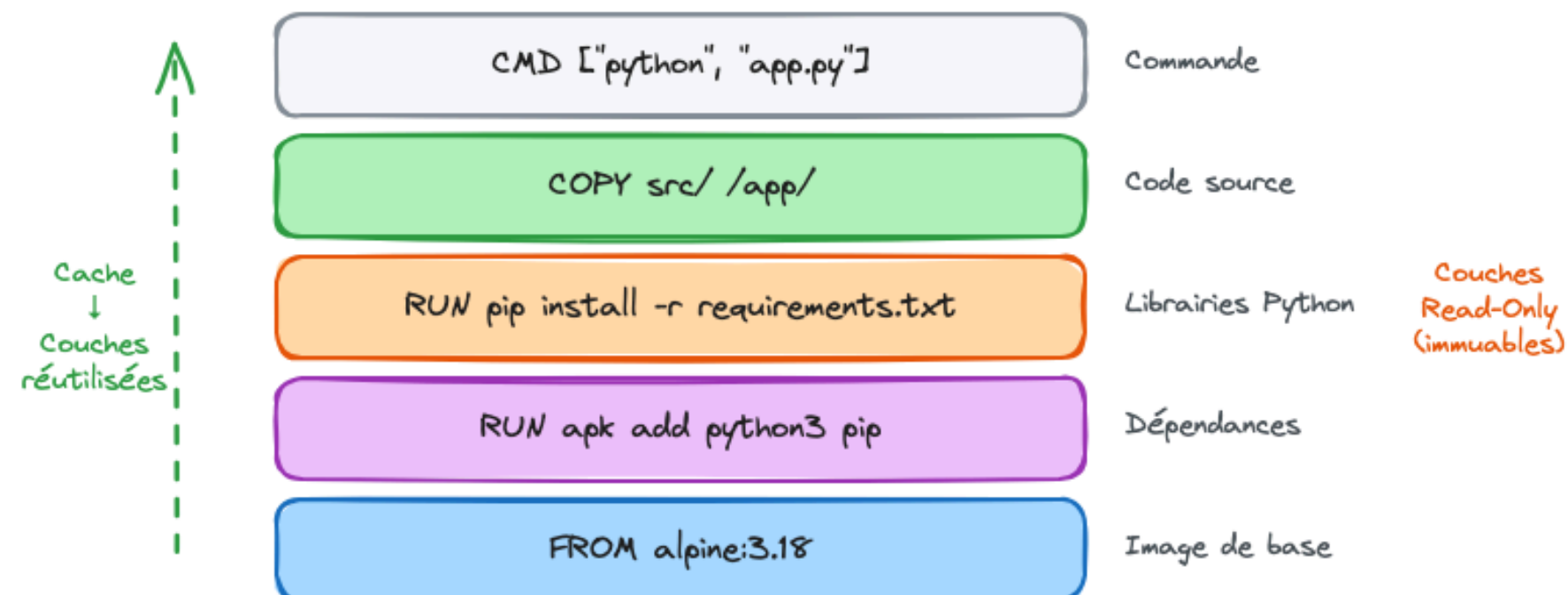
Objets Docker

- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins

Les Images Docker

- Une image est :
 - Un modèle en lecture seule
 - Une suite d'instructions
 - Une base pour créer des conteneurs
- Exemple :
 - Image Ubuntu
 - Image Python
 - Image Nginx

Couches d'une image Docker



Concepts Fondamentaux

Objets Docker

- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins

Structure d'une Image

- Une image est composée de :
 - Plusieurs couches (layers)
 - Chaque instruction Dockerfile crée une couche
- Exemple :
 - **FROM ubuntu**
 - **RUN apt install python**
 - **COPY app.py**

Chaque ligne crée une nouvelle couche.

Les Images Docker

- Une image est :
 - Un modèle en lecture seule
 - Une suite d'instructions
 - Une base pour créer des conteneurs
- Exemple :
 - Image Ubuntu
 - Image Python
 - Image Nginx
- Pour créer une image :
 - Écrire un Dockerfile
 - Définir les étapes de construction
 - Construire avec :
 - **docker build -t monimage .**
- Seules les couches modifiées sont reconstruites.

Concepts Fondamentaux

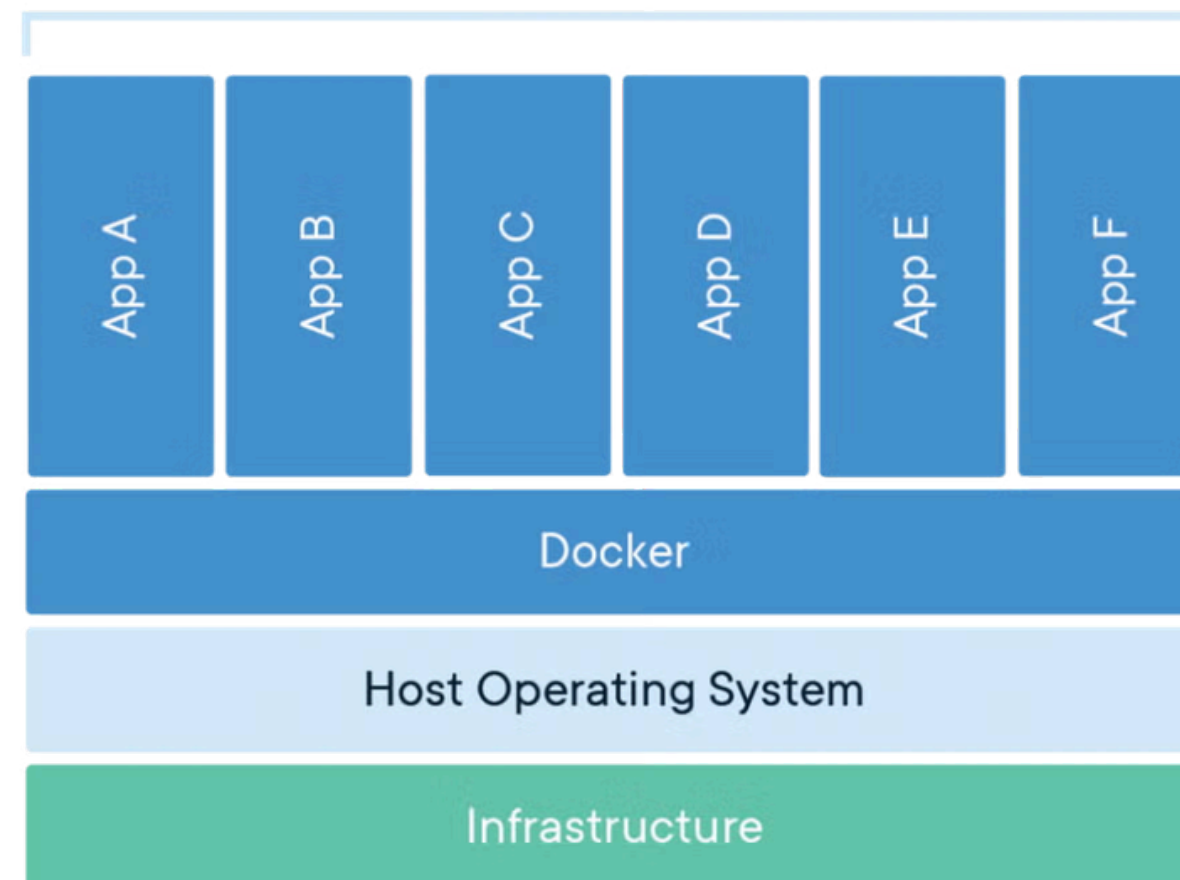
Objets Docker

- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins

Les Conteneurs

- Un conteneur est :
 - Une instance exécutable d'une image
 - Un processus isolé
 - Un environnement autonome
- On peut :
 - Créer, Démarrer, Arrêter, Supprimer, Déplacer

Containerized Applications





Concepts Fondamentaux

Objets Docker

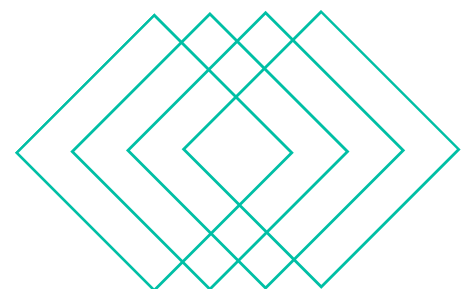
- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins

Isolation d'un Conteneur

- Par défaut, un conteneur est isolé :
 - Réseau
 - Système de fichiers
 - Processus
 - Ressources
- Mais on peut contrôler :
 - Réseau partagé
 - Volumes montés
 - Limites CPU/RAM

Les Conteneurs

- Un conteneur est :
 - Une instance exécutable d'une image
 - Un processus isolé
 - Un environnement autonome
- On peut :
 - Créer, Démarrer, Arrêter, Supprimer, Déplacer





Concepts Fondamentaux

Objets Docker

- Docker manipule plusieurs types d'objets :
 - Images
 - Conteneurs
 - Réseaux
 - Volumes
 - Plugins

Isolation d'un Conteneur

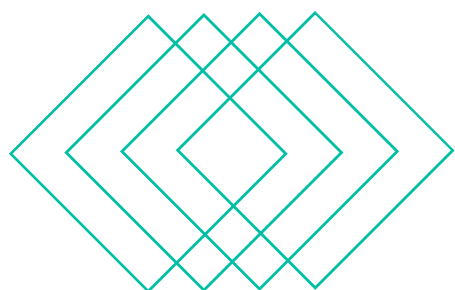
- Par défaut, un conteneur est isolé :
 - Réseau
 - Système de fichiers
 - Processus
 - Ressources
- Mais on peut contrôler :
 - Réseau partagé
 - Volumes montés
 - Limites CPU/RAM

Les Conteneurs

- Un conteneur est :
 - Une instance exécutable d'une image
 - Un processus isolé
 - Un environnement autonome
- On peut :
 - Créer, Démarrer, Arrêter, Supprimer, Déplacer

Cycle de Vie d'un Conteneur

- Étapes :
 - Création, Démarrage
 - Exécution
 - Arrêt
 - Suppression
- Commande clé :
 - **docker run**
 - **docker stop**
 - **docker rm**

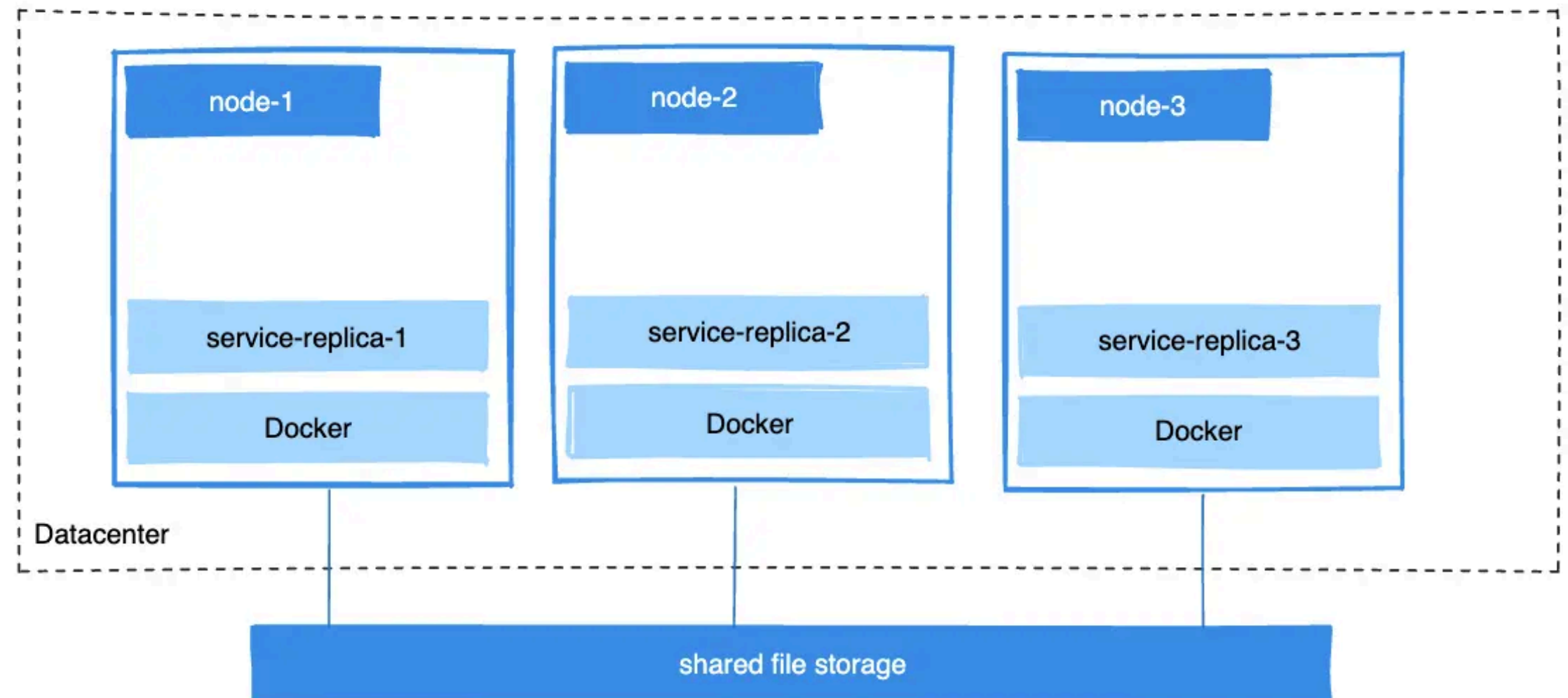


Concepts Fondamentaux

Objets Docker

Persistance des Données

- Par défaut :
 - Si un conteneur est supprimé → données perdues.
- Utiliser des volumes
- Les volumes permettent :
 - Persistance
 - Partage entre conteneurs
 - Sauvegarde des données



Commandes Essentielles & Manipulation

Images : docker pull

- Télécharger une image
 - **docker pull nginx:alpine**
- Ce qui se passe :
 - Client Docker
 - ↓
 - Requête au registry (Docker Hub)
 - ↓
 - Téléchargement des layers
 - ↓
 - Stockage local

Important :

- Les images sont composées de layers
- Les layers déjà présents ne sont pas re-téléchargés

Image A

- Base OS
- Ubuntu
- Python
- App A

docker pull Image_A

Image B

- Base OS
- Ubuntu
- NodeJs
- App B

docker pull Image_B

Commandes Essentielles & Manipulation

Voir les images locales

- **docker images**
- Affiche :
 - REPOSITORY, TAG, IMAGE ID, SIZE

```
REPOSITORY    TAG         IMAGE ID      CREATED        SIZE
optilevel     latest      ff1241785782  2 months ago  1.26GB
node          20-alpine   eabac870db94  5 months ago  190MB
mariadb       11.3        e101f9db3191  21 months ago 528MB
```

Supprimer une image

- **docker rmi <nom_images>**
- Impossible si un conteneur l'utilise.



Commandes Essentielles & Manipulation

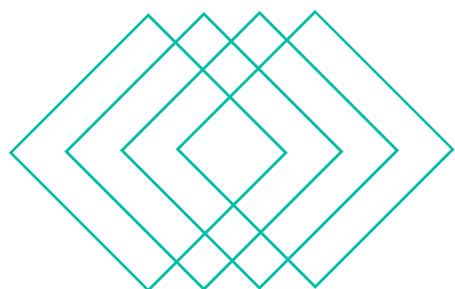


Conteneurs : docker run

- **docker run <nom_image>**
- Ce que Docker fait :
 - 1. Vérifie si l'image existe
 - 2. Sinon → docker pull
 - 3. Crée un conteneur
 - 4. Lance le process principal

Les paramètres de la commande docker run

- -it: interactif + TTY (pseudo terminal)
- -d: détaché (background)
- --name: nom du conteneur
- -p: publier un port
- -v: monter un volume (docker run -v data:/file ubuntu)
- -e: Variables d'environnement



Commandes Essentielles & Manipulation

Conteneurs : docker run

- **docker run <nom_image>**
- Ce que Docker fait :
 - 1. Vérifie si l'image existe
 - 2. Sinon → docker pull
 - 3. Crée un conteneur
 - 4. Lance le process principal

Les paramètres de la commande docker run

- -it: interactif + TTY (pseudo terminal)
- -d: détaché (background)
- --name: nom du conteneur
- -p: publier un port
- -v: monter un volume (docker run -v data:/file ubuntu)
- -e: Variables d'environnement

Publier un port

- docker run -d -p 8080:80 nginx
- Schéma :
 - Machine : 8080 → Conteneur : 80
- Port hôte : 8080
- Port conteneur : 80

Sans -p → inaccessible depuis l'extérieur

Machine Hôte
curl http://localhost:8080 ?
curl http://localhost:80 ?

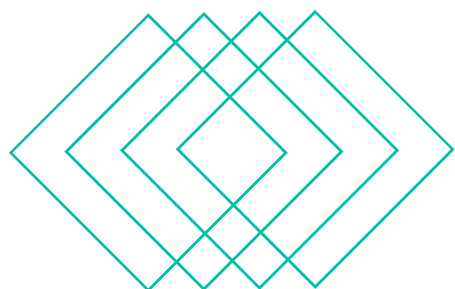
Conteneur
serveur nginx qui
tourne sur le port 80



Commandes Essentielles & Manipulation

docker ps

- docker ps
- docker ps -a
- Affiche:
 - ID
 - IMAGE
 - STATUS
 - PORTS
 - NAMES





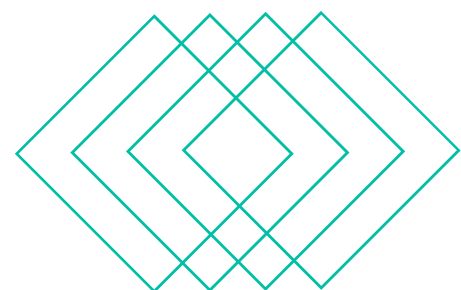
Commandes Essentielles & Manipulation

docker ps

- docker ps
- docker ps -a
- Affiche:
 - ID
 - IMAGE
 - STATUS
 - PORTS
 - NAMES

docker stop / start / rm

- docker stop <nom_conteneur>
- docker start <nom_conteneur>
- docker rm <nom_conteneur>





Commandes Essentielles & Manipulation

docker ps

- docker ps
- docker ps -a
- Affiche:
 - ID
 - IMAGE
 - STATUS
 - PORTS
 - NAMES

docker stop / start / rm

- docker stop <nom_conteneur>
- docker start <nom_conteneur>
- docker rm <nom_conteneur>

Debug & Inspection

- docker logs web
- docker logs -f web

Affiche stdout / stderr du process principal.

Commandes Essentielles & Manipulation

docker ps

- docker ps
- docker ps -a
- Affiche:
 - ID
 - IMAGE
 - STATUS
 - PORTS
 - NAMES

docker stop / start / rm

- docker stop <nom_conteneur>
- docker start <nom_conteneur>
- docker rm <nom_conteneur>

Debug & Inspection

- docker logs web
- docker logs -f web

Affiche stdout / stderr du process principal.

docker exec

- **docker exec -it <nom_conteneur> bash**
- Terminal
 - ↓
- Entrée dans le conteneur déjà en cours

Commandes Essentielles & Manipulation

docker ps

- docker ps
- docker ps -a
- Affiche:
 - ID
 - IMAGE
 - STATUS
 - PORTS
 - NAMES

docker stop / start / rm

- docker stop <nom_conteneur>
- docker start <nom_conteneur>
- docker rm <nom_conteneur>

Debug & Inspection

- docker logs web
- docker logs -f web

Affiche stdout / stderr du process principal.

docker exec

- **docker exec -it <nom_conteneur> bash**
- Terminal
 - ↓
- Entrée dans le conteneur déjà en cours

docker inspect

- Affiche:
 - Réseau
 - IP
 - Variables d'environnement
 - Volumes
 - Configuration complète (JSON)



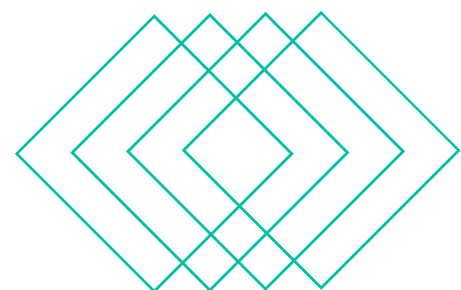
Commandes Essentielles & Manipulation

docker stats

- Affiche en temps réel
 - CPU %
 - Mémoire
 - Réseau
 - I/O disque

docker top

- Montre :
 - Processus actifs dans le conteneur





Commandes Essentielles & Manipulation

docker stats

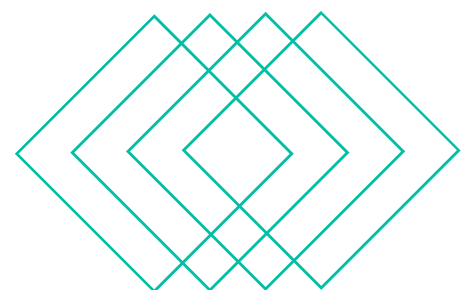
- Affiche en temps réel
 - CPU %
 - Mémoire
 - Réseau
 - I/O disque

docker top

- Montre :
 - Processus actifs dans le conteneur

Gestion des ressources

- Limiter CPU
 - **docker run --cpus="0.5" ubuntu**
 - 50% d'un CPU maximum
- Limiter Mémoire
 - **docker run -m 256m ubuntu**
 - Limite RAM à 256 Mo

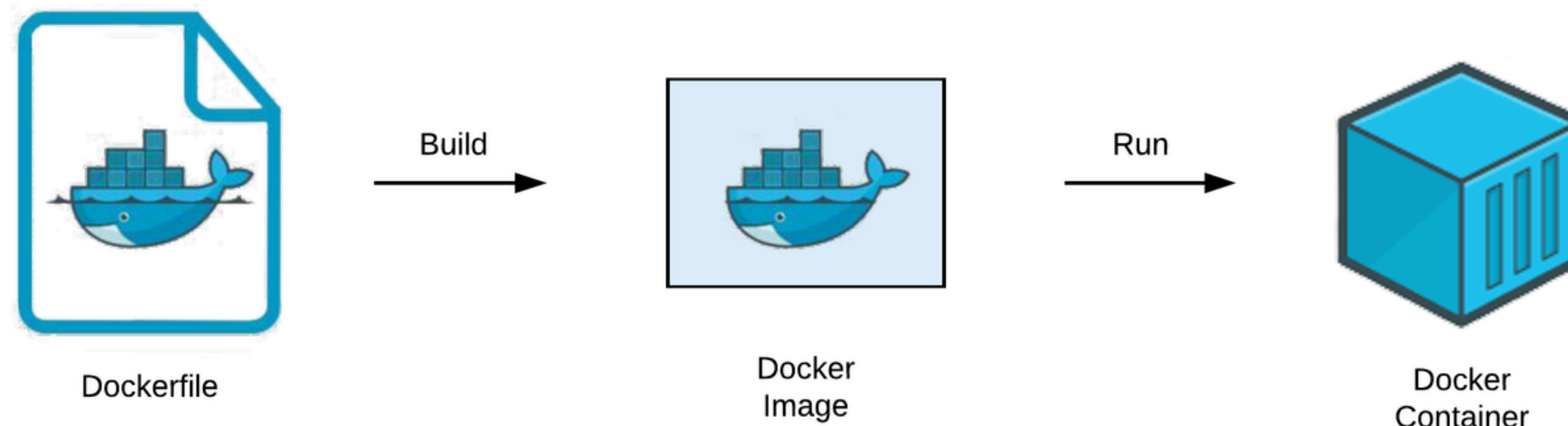


Dockerfile & Construction d'Images

- Un Dockerfile est :
 - Un fichier texte
 - Une suite d'instructions
 - Une recette pour construire une image
- Chaque instruction crée un layer

Exemple simple

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY ..
EXPOSE 8000
CMD ["python", "app.py"]
```





Dockerfile & Construction d'Images

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

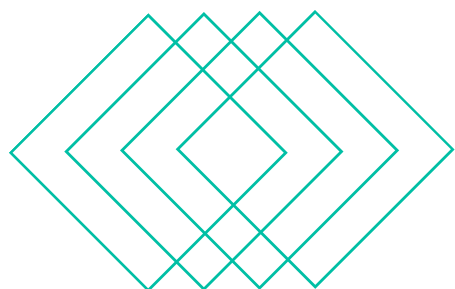
COPY ..

EXPOSE 8000

CMD ["python", "app.py"]

FROM ...

- Définit l'image de base
- Toujours première instruction
- Peut être :
 - ubuntu
 - python
 - node
 - nginx
 - ...
- Hérite des layers de l'image de base





Dockerfile & Construction d'Images

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

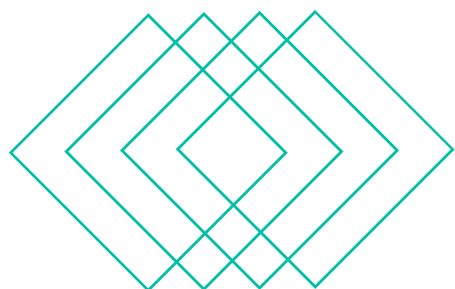
COPY ..

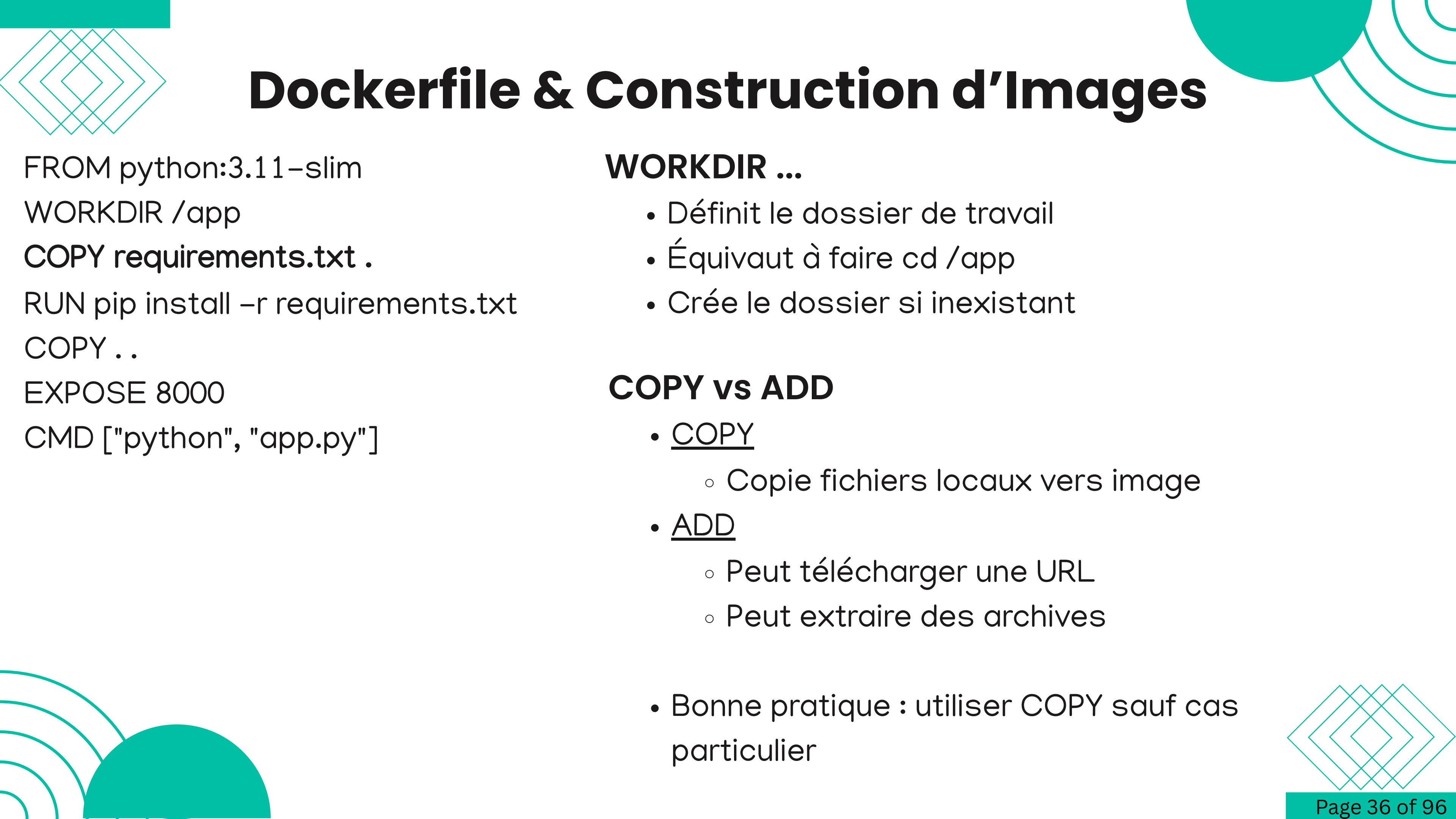
EXPOSE 8000

CMD ["python", "app.py"]

WORKDIR ...

- Définit le dossier de travail
- Équivaut à faire `cd /app`
- Crée le dossier si inexistant





Dockerfile & Construction d'Images

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY ..

EXPOSE 8000

CMD ["python", "app.py"]

WORKDIR ...

- Définit le dossier de travail
- Équivaut à faire `cd /app`
- Crée le dossier si inexistant

COPY vs ADD

- COPY
 - Copie fichiers locaux vers image
- ADD
 - Peut télécharger une URL
 - Peut extraire des archives
- Bonne pratique : utiliser COPY sauf cas particulier

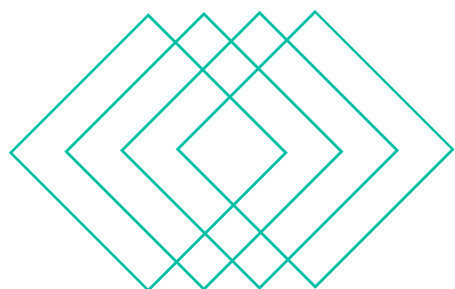


Dockerfile & Construction d'Images

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY ..
EXPOSE 8000
CMD ["python", "app.py"]
```

RUN ...

- Exécuté pendant le build
- Crée un nouveau layer
- Sert à installer dépendances





Dockerfile & Construction d'Images

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY ..

EXPOSE 8000

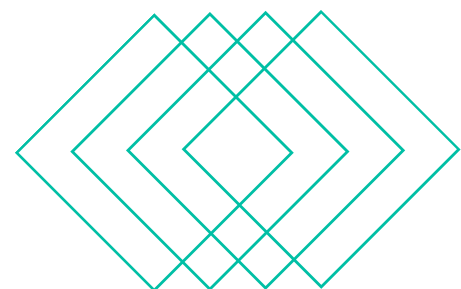
CMD ["python", "app.py"]

RUN ...

- Exécuté pendant le build
- Crée un nouveau layer
- Sert à installer dépendances

CMD vs ENTRYPOINT

- CMD
 - CMD ["python", "app.py"]
 - Commande par défaut
 - Peut être remplacée au run
- ENTRYPOINT
 - ENTRYPOINT ["python"]
 - CMD ["app.py"]
 - Définit le programme principal
 - CMD devient argument





Dockerfile & Construction d'Images

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY ..
EXPOSE 8000
CMD ["python", "app.py"]
```

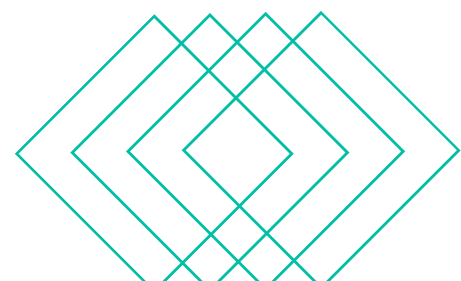
EXPOSE ...

- Ne publie PAS le port
 - C'est une documentation
 - -p publie réellement le port
- 

RUN ...

- Exécuté pendant le build
- Crée un nouveau layer
- Sert à installer dépendances

CMD vs ENTRYPOINT

- CMD
 - CMD ["python", "app.py"]
 - Commande par défaut
 - Peut être remplacée au run
 - ENTRYPOINT
 - ENTRYPOINT ["python"]
 - CMD ["app.py"]
 - Définit le programme principal
 - CMD devient argument
- 

Dockerfile & Construction d'Images

Construire une image

- `docker build -t monapp:1.0 .`
- Ce qui se passe :
 - 1. Docker envoie le build context
 - 2. Exécute chaque instruction
 - 3. Crée un layer par instruction
 - 4. Génère une image finale

Layers & Cache

- Chaque instruction = 1 layer
- Si rien ne change → layer réutilisé

Mauvais ordre (erreur classique)

- `COPY ..`
- `RUN pip install -r requirements.txt`

Si tu modifies un fichier :

- `pip install` est relancé
- Build lent



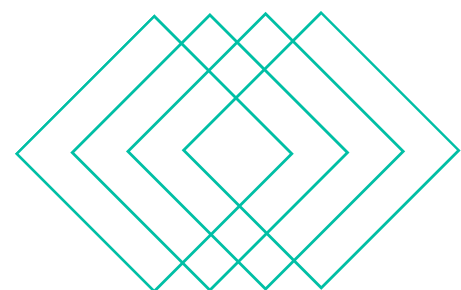
Dockerfile & Construction d'Images

.dockerignore

- Permet d'éviter d'envoyer :
 - node_modules
 - .git
 - venv
 - fichiers temporaires
- Build plus rapide
- Image plus légère

Exemple

```
__pycache__/  
.venv/  
.git/  
node_modules/
```





Dockerfile & Construction d'Images

Multi-stage Build (Concept)

- Permet :
 - Construire dans une image lourde
 - Copier uniquement le résultat final
 - Image finale plus petite

Exemple

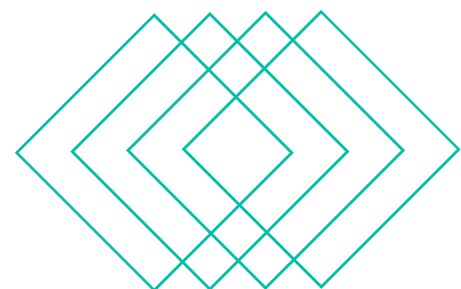
```
FROM node AS build  
RUN npm build
```

```
FROM nginx
```

```
COPY --from=build /app/dist /usr/share/nginx/html
```

Problème

- Quand on build une application frontend (React, Vue, Angular...) :
- On a besoin de Node.js
- On installe plein de dépendances
- On compile le code
- Mais en production:
 - On n'a PAS besoin de Node
 - On a juste besoin des fichiers générés (HTML/CSS/JS)





Dockerfile & Construction d'Images

Multi-stage Build (Concept)

- Permet :
 - Construire dans une image lourde
 - Copier uniquement le résultat final
 - Image finale plus petite

Exemple Sans Multi-Stage

FROM node

WORKDIR /app

COPY ..

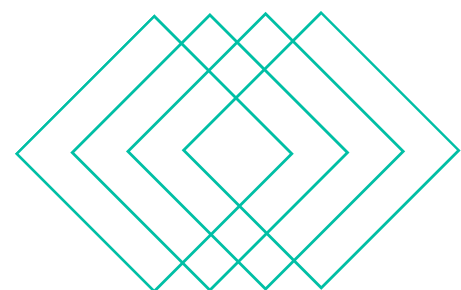
RUN npm install

RUN npm run build

- Node reste dans l'image finale
- node_modules reste
- Image énorme (400-800MB)
- Moins sécurisée

Problème

- Quand on build une application frontend (React, Vue, Angular...) :
- On a besoin de Node.js
- On installe plein de dépendances
- On compile le code
- Mais en production:
 - On n'a PAS besoin de Node
 - On a juste besoin des fichiers générés (HTML/CSS/JS)





Dockerfile & Construction d'Images

Exemple avec Multi-stage

FROM node AS build

RUN npm build

FROM nginx

COPY --from=build /app/dist

/usr/share/nginx/html

FROM node AS build

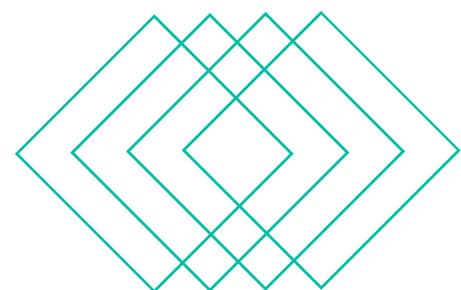
- On utilise l'image Node
- On lui donne un nom : build

C'est une "étape temporaire"

RUN npm build

- On compile l'application
- Ça génère un dossier (ex: /app/dist)

- À ce stade :
 - Node est nécessaire
 - Dépendances sont installées
 - Fichiers finaux sont générés



Dockerfile & Construction d'Images

Exemple avec Multi-stage

FROM node AS build

RUN npm build

FROM nginx

COPY --from=build /app/dist
/usr/share/nginx/html

FROM nginx

- Nouvelle image
- Nouvelle base propre
- Node disparaît complètement

Copier uniquement le résultat

COPY --from=build /app/dist /usr/share/nginx/html

- --from=build → prend les fichiers depuis l'étape précédente
- /app/dist → dossier compilé
- /usr/share/nginx/html → dossier servi par nginx

On copie seulement le résultat final

Dockerfile & Construction d'Images

Étape 1 (build)

- node image
- npm install
- npm build
- → dist/

Étape 2 (production)

- nginx image
- copie dist/
- sert fichiers

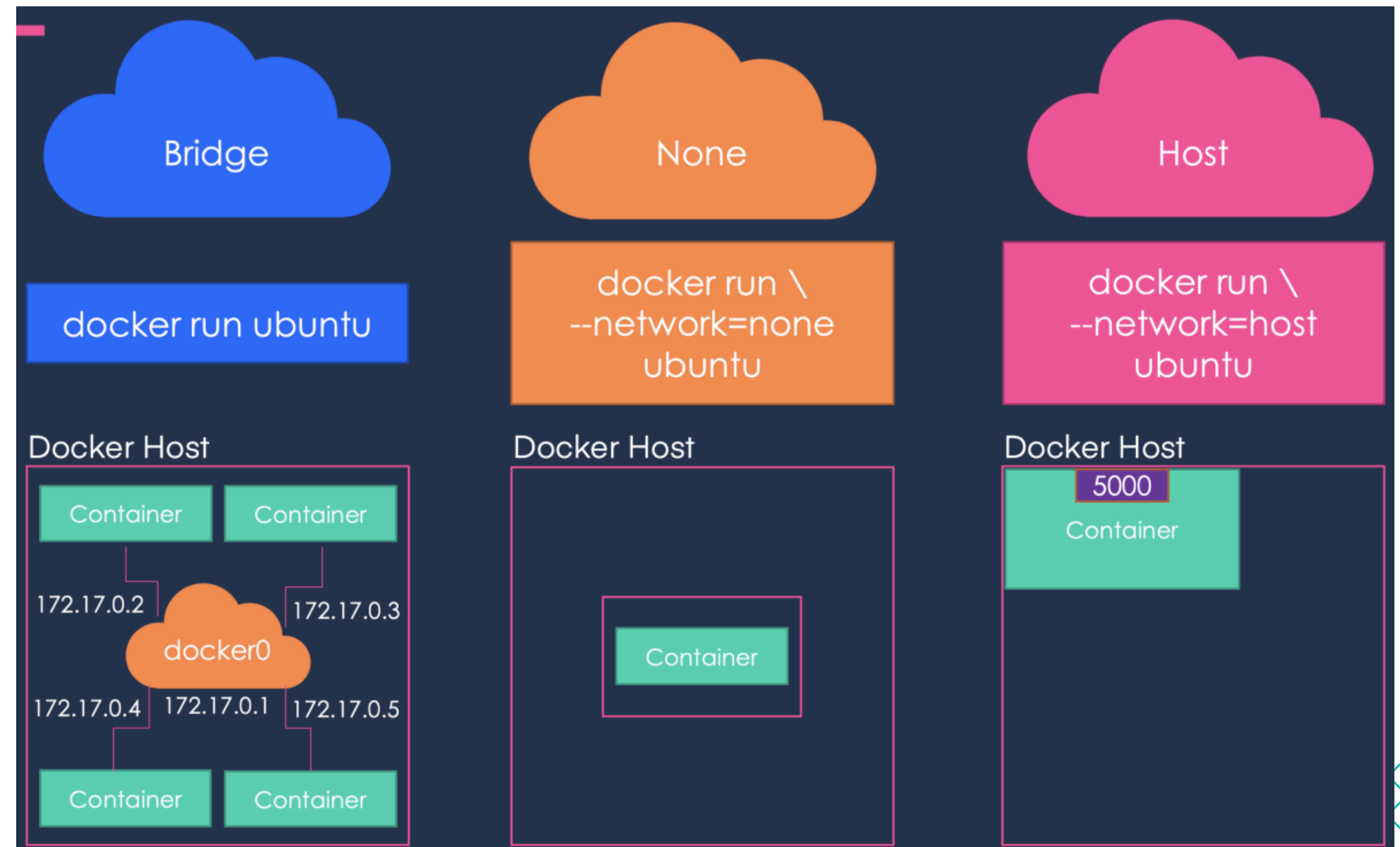
Image finale contient :

- nginx
- fichiers compilés
- pas node
- pas node_modules
- pas outils de build
- Image beaucoup plus petite
- Plus sécurisée
- Plus rapide à déployer

Réseaux Docker

Pourquoi les réseaux Docker existent ?

- Par défaut :
 - Un conteneur est isolé
 - Il n'a pas accès direct aux autres conteneurs
- Docker crée un réseau virtuel pour :
 - Isoler
 - Connecter
 - Sécuriser



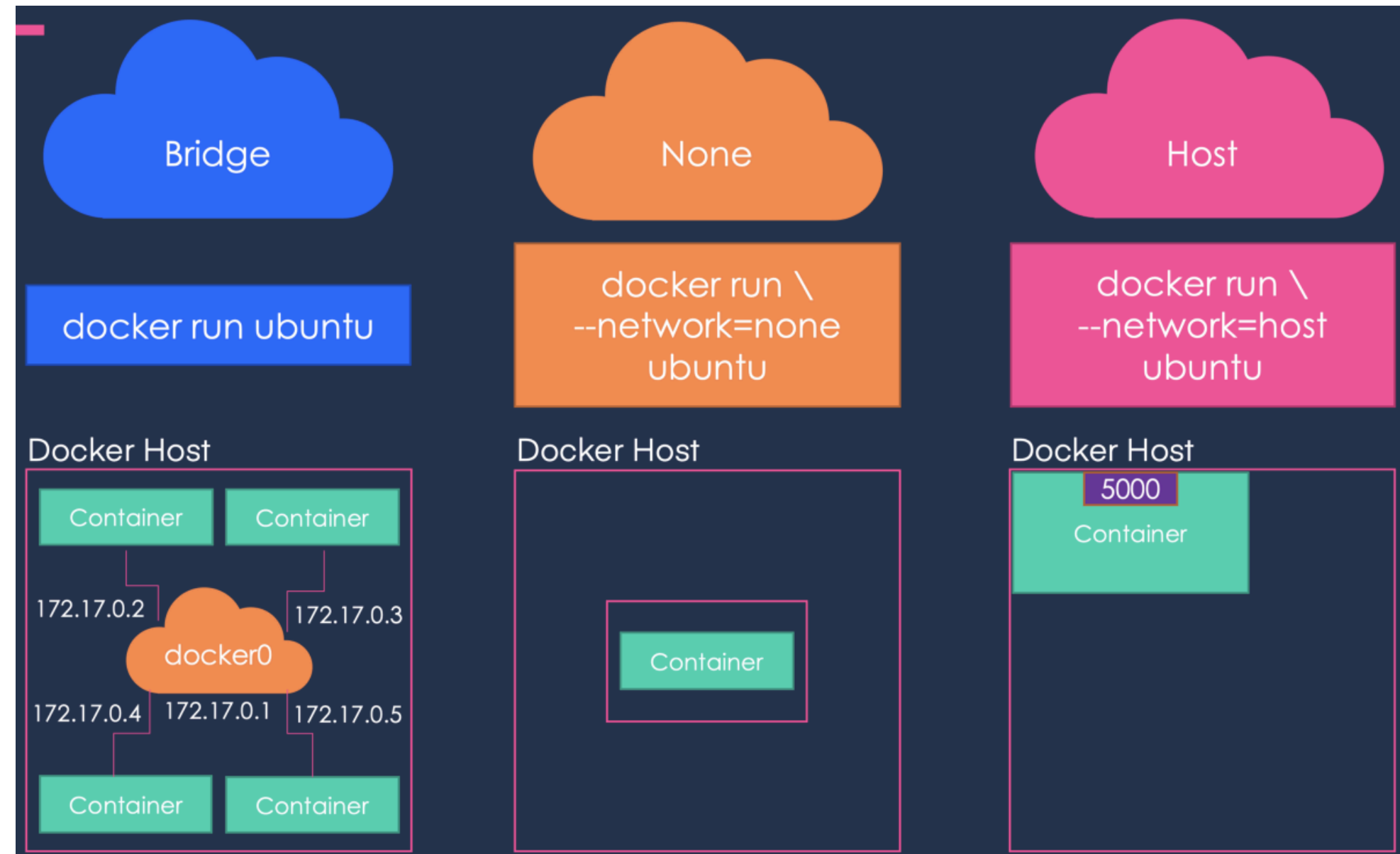
Réseaux Docker

Docker propose plusieurs modes réseau :

- bridge (par défaut)
- none
- host
- user-defined bridge (réseau personnalisé)

Chaque mode change :

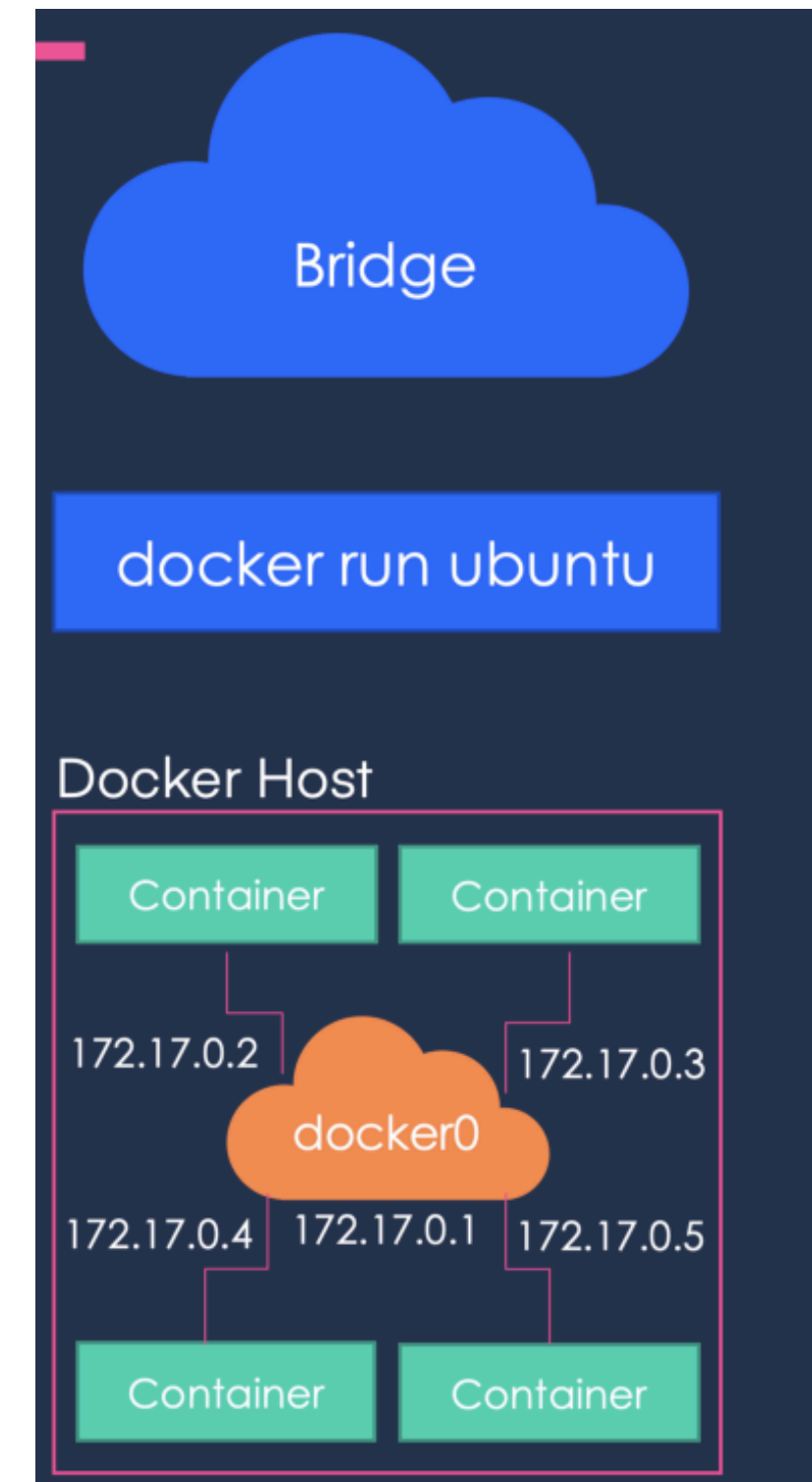
- L'isolation
- La communication
- L'accès aux ports



Réseaux Docker

Mode Bridge (par défaut)

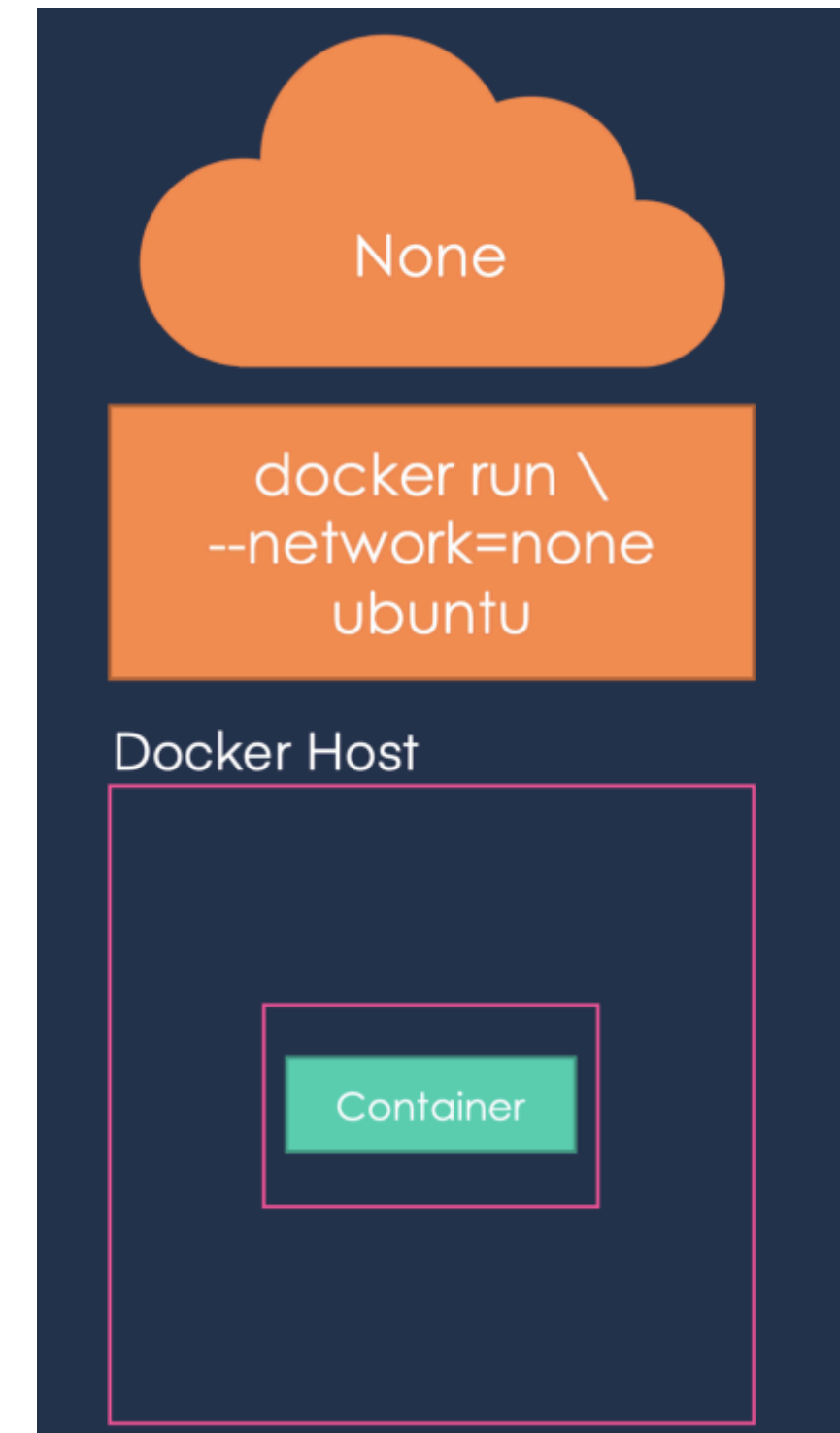
- Docker connecte le conteneur au réseau :
 - bridge (docker0)
 - IP interne automatique
 - Isolation réseau
 - Pas accessible depuis l'extérieur sans -p
-
- Sans publication du port (-p) :
 - Machine → inaccessible
 - Avec -p :
 - **docker run -p 8080:80 ubuntu**
-
- Navigateur
 - ↓
 - localhost:8080
 - ↓
 - Port 80 du conteneur



Réseaux Docker

Mode None

- Isolation totale
- Pas d'IP
- Pas d'accès internet
- Pas de communication
- Cas d'usage :
 - Sécurité
 - Traitement offline



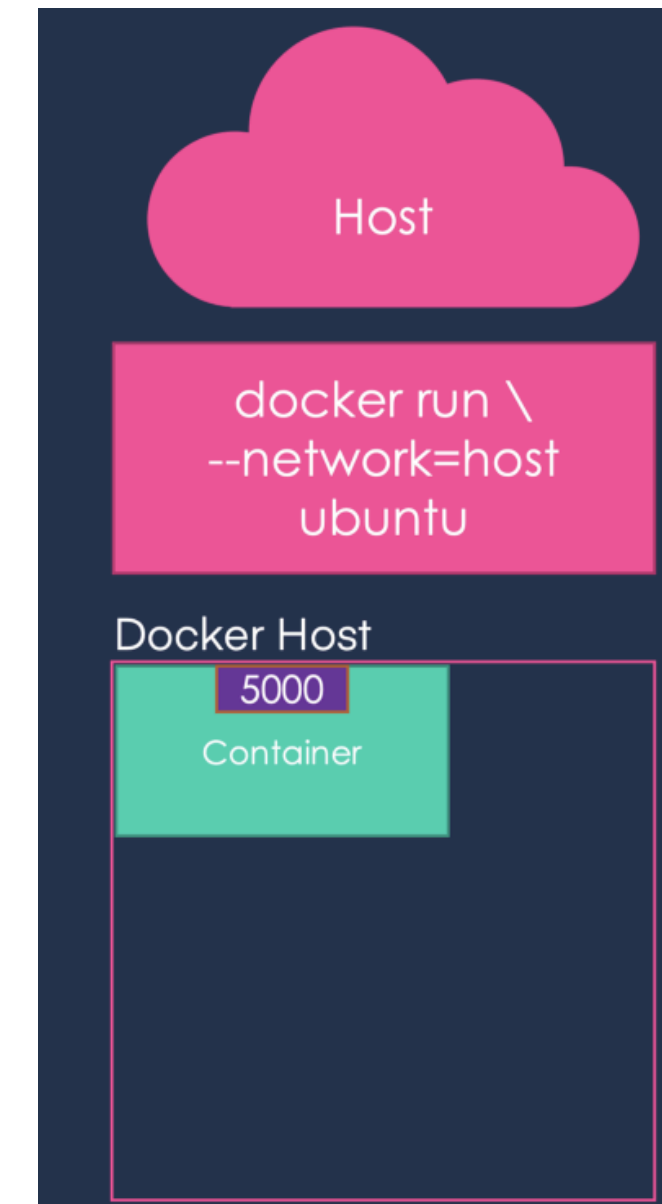
Réseaux Docker

Mode Host

- Le conteneur utilise directement le réseau de la machine
- Pas d'isolation réseau
- Pas besoin de -p
- Performance maximale
- Pas d'isolation réseau

Le conteneur :

- Utilise les ports de la machine directement
- Si l'app écoute sur 5000 :
 - → elle écoute sur la machine 5000

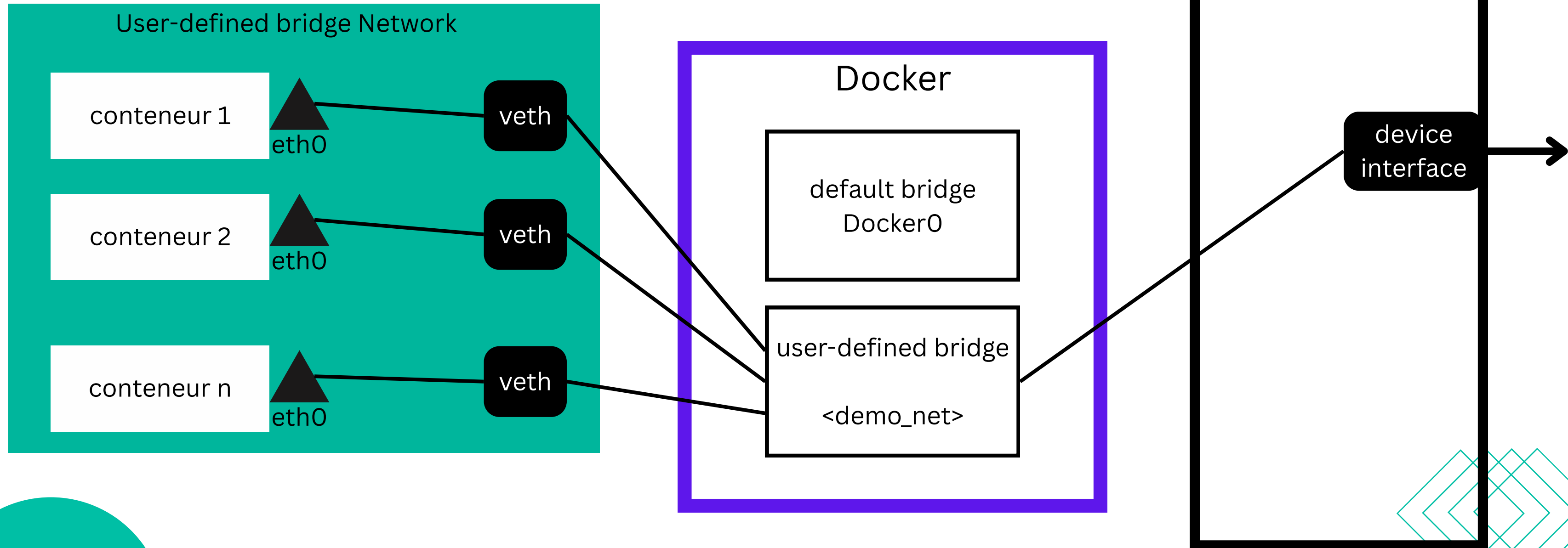


**En mode host, le conteneur ne passe plus par docker0.
Il est directement branché sur la carte réseau de la machine.**

Réseaux Docker

Réseau personnalisé (important)

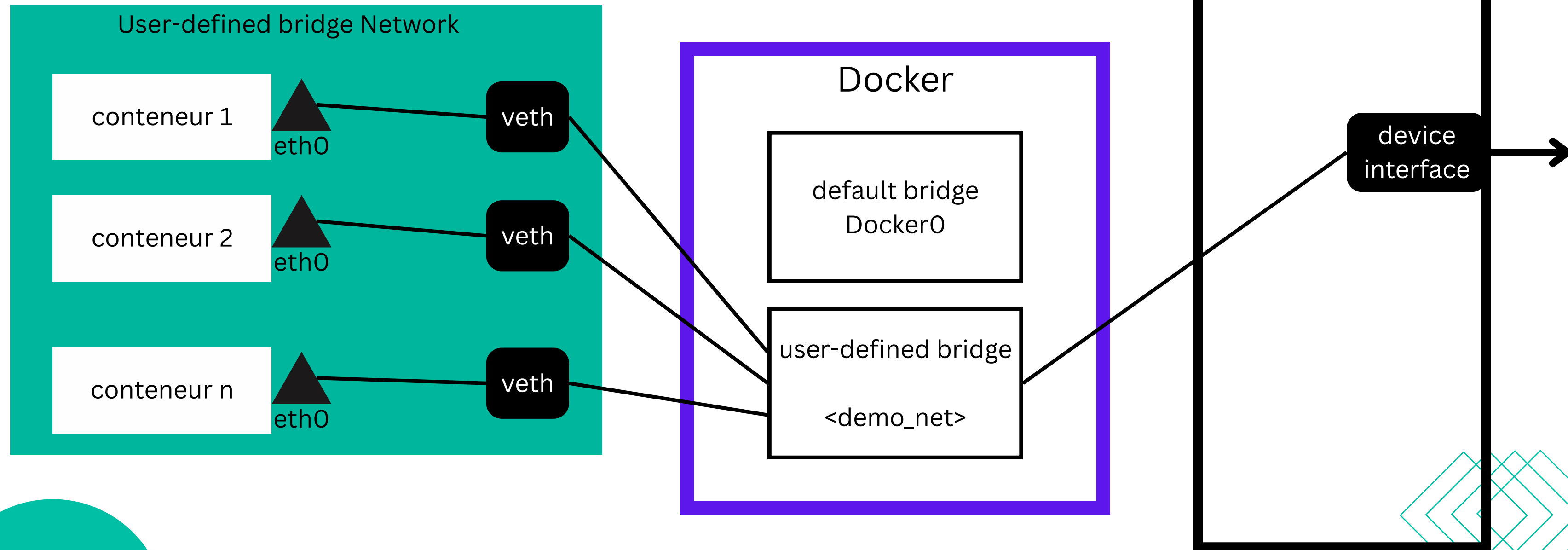
- Créer un réseau :
 - **docker network create <nom_réseau>**
- Puis lancer :
 - **docker run --network <nom_réseau> --name <nom_conteneur> ...**



Réseaux Docker

Réseau personnalisé (important)

- Communication par nom
 - conteneur 1 → curl http://conteneur 2:port
 - Pas besoin d'IP !
- Docker résout automatiquement nom_conteneur → IP (DNS interne)



Réseaux Docker

Inspecter un réseau

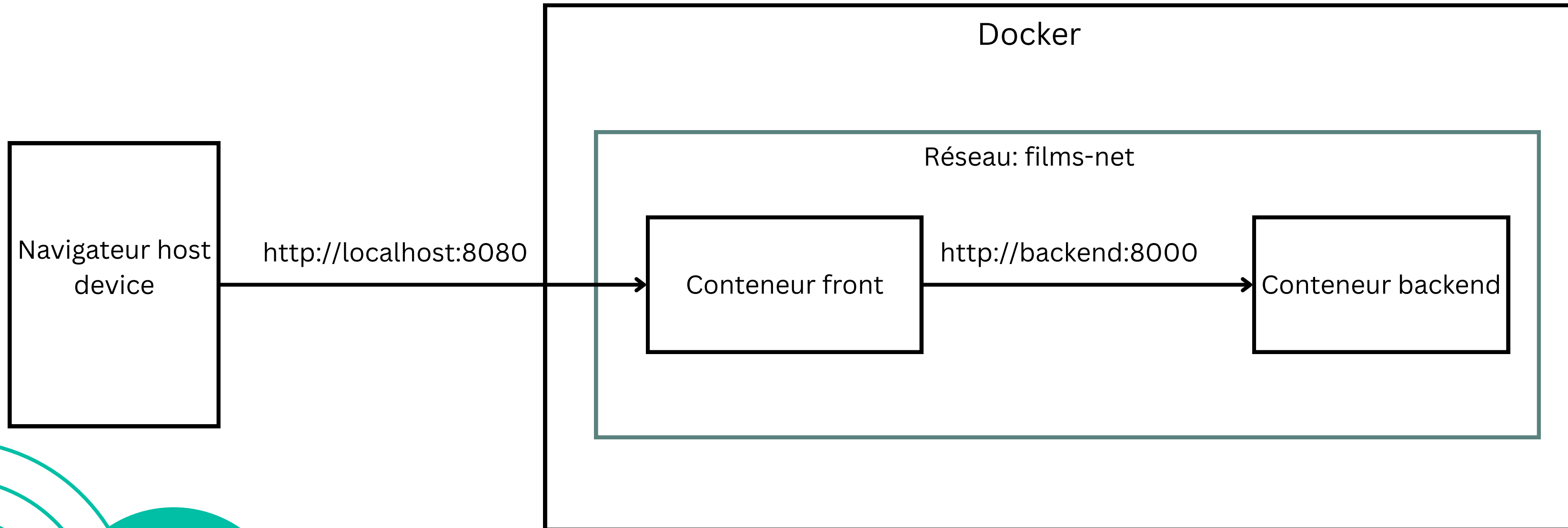
- **docker network ls**
- **docker network inspect <nom_réseaux>**
- Permet de voir :
 - Conteneurs connectés
 - IP
 - Subnet

Debug réseau

- Si ça ne marche pas :
 - Vérifier si même réseau
 - Vérifier ports
 - Vérifier que l'API écoute sur 0.0.0.0
 - Tester depuis conteneur :
 - `docker exec -it <conteneur_1> sh`
 - `curl http://<conteneur_2>:port` (Si API ou serveur)
 - `ping <conteneur_2>`

Réseaux Docker

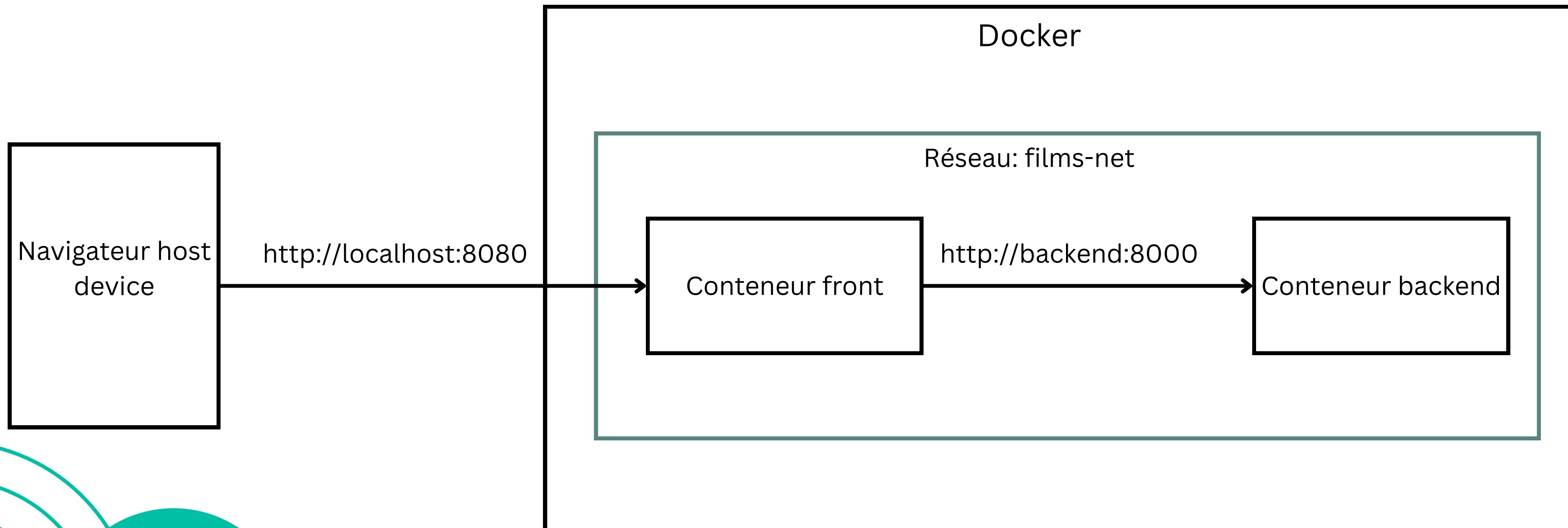
Exemple



Réseaux Docker

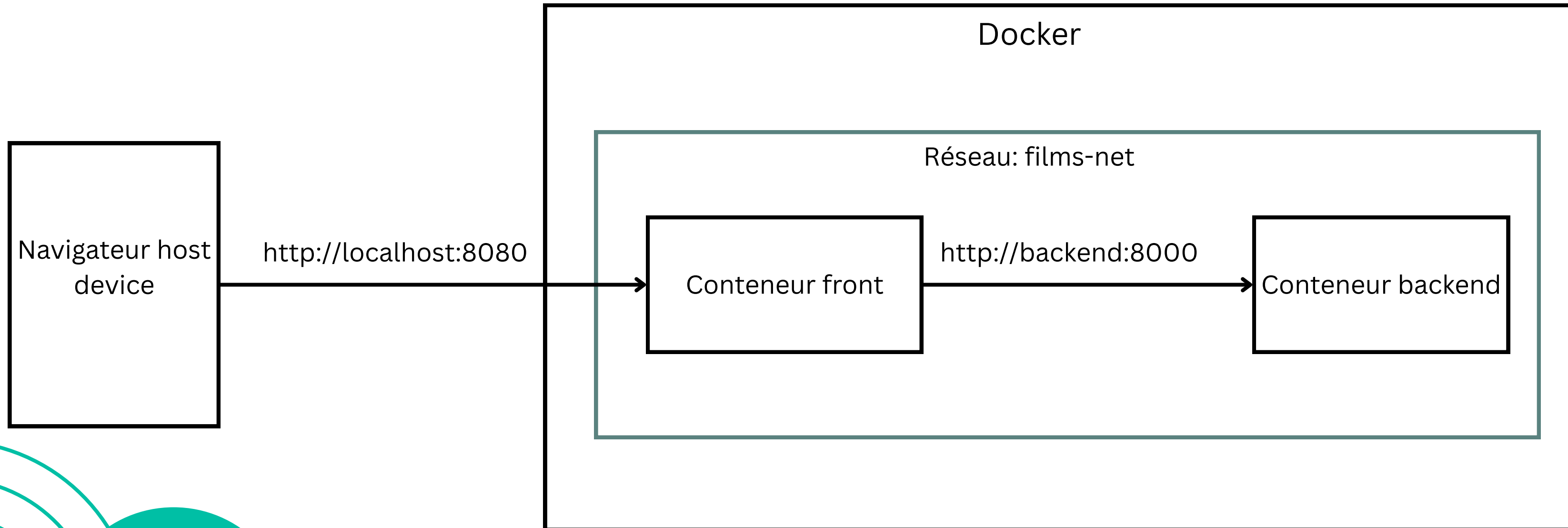
Exemple

- `docker network create films-net`
- `docker run -d --name backend --network films-net -p 8000:8000 my-backend-image`
- `docker run -d --name frontend --network films-net -p 8080:80 my-frontend-image`



Réseaux Docker

- Si dans le frontend on écrit :
 - `fetch("http://localhost:8000")`
- Ça marche ?
- Pourquoi ?



Réseaux Docker

En production ou en projet multi-services,
on n'utilise presque jamais le bridge par
défaut.

On crée toujours un réseau personnalisé.

Dans Docker, on ne communique
pas avec localhost.

On communique avec le nom du
conteneur.

Mode	Isolation	DNS interne	Publication requis
bridge	Oui	Non	Oui
none	Totale	Non	Non
host	Non	Non	Non
user-defined	Oui	Oui	Oui

- Ce qu'est le réseau bridge
- Publier un port
- Créer un réseau personnalisé
- Utiliser les noms DNS
- Debugger un problème réseau



Volumes & Persistance

Problème : données perdues

- Par défaut :
 - Un conteneur possède une couche writable temporaire
- Si on supprime le conteneur :
 - **docker rm monconteneur**
- Les fichiers créés à l'intérieur disparaissent

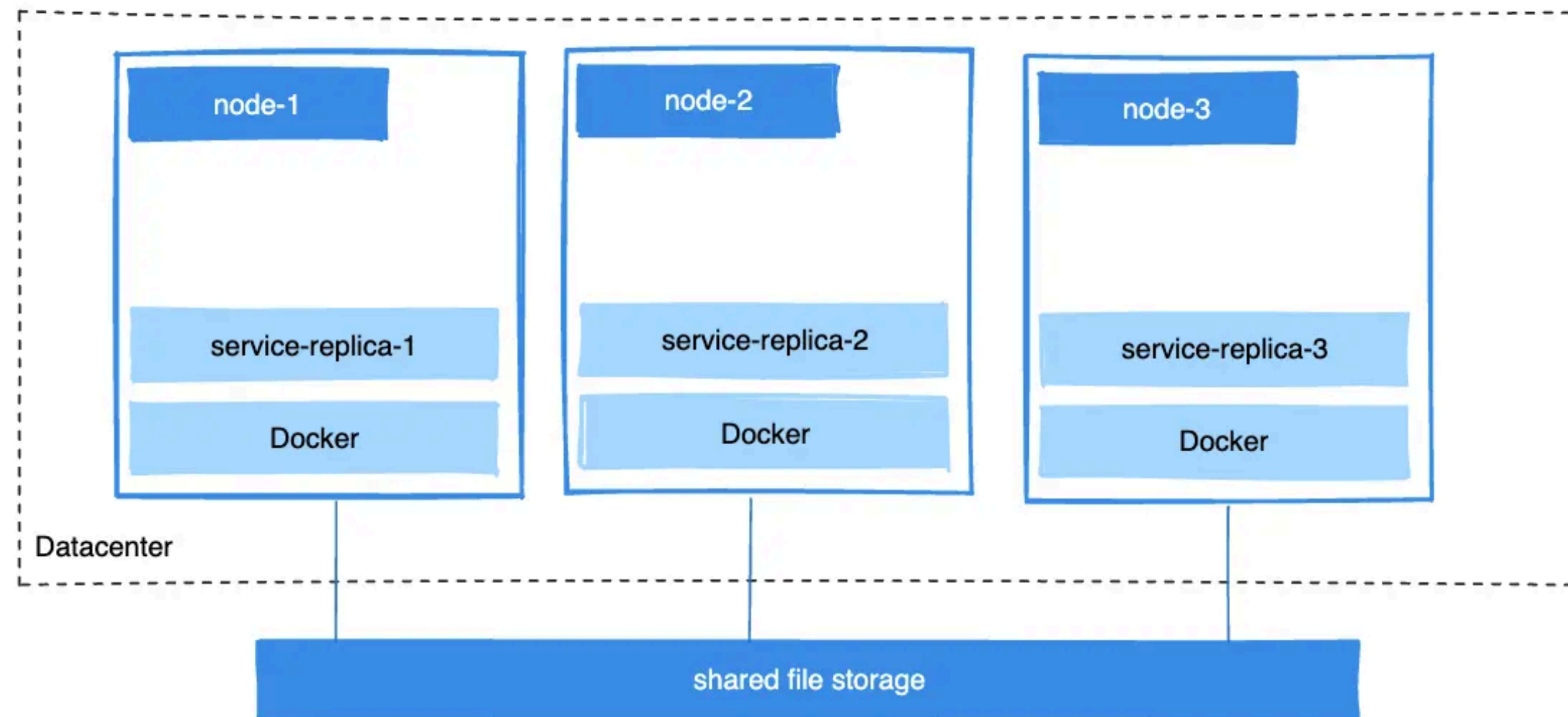
Exemple

- docker run -it ubuntu bash
- touch test.txt
- exit
- docker rm <container>
- docker run -it ubuntu bash

Le fichier test.txt n'existe plus.

Volumes & Persistence

- Un volume est :
 - Un espace de stockage externe au conteneur
 - Géré par Docker
 - Persistant



Volumes & Persistance

- Créer un Volume
 - `docker volume create data_volume`
- Voir les volumes :
 - `docker volume ls`
- Utiliser un Volume
 - `docker run -v data_volume:/data ubuntu`
- `data_volume` → volume Docker
- `/data` → dossier dans le conteneur
- Test
 - `docker run -it --name test -v data_volume:/data ubuntu`
 - `touch /data/file.txt`
 - `exit`
 - `docker rm test`
 - `docker run -it -v data_volume:/data ubuntu`
 - `ls /data`
- Le fichier existe ?

Volumes & Persistance

Bind Mount

- Bind mount = lier un dossier de la machine au conteneur.
- Exemple:
 - `docker run -v $(pwd):/app ubuntu`

<u>Volume Docker</u>	<u>Bind Mount</u>
Géré par Docker	Géré par l'OS
Plus portable	Dépend du chemin local
Idéal production	Idéal développement

Volumes & Persistance

Où sont stockés les volumes ?

Linux :

- `/var/lib/docker/volumes/`

Mais on ne manipule pas ça directement.

- Supprimer un Volume
 - `docker volume rm data_volume`
- Supprime définitivement les données

• Quand utiliser quoi ?

- Volume nommé
 - Base de données
 - Production
 - Persistance
- Bind mount
 - Développement
 - Synchroniser code
 - Live reload

Docker Compose

Sans Docker Compose

Exemple :

- Front + Back + Base de données

Sans Compose :

- docker network create my-net
- docker volume create pgdata
- docker run ...
- docker run ...
- docker run ...
- ...

- Beaucoup de commandes
- Complexe
- Facile à oublier une option

Docker Compose

Avec Docker Compose `docker compose up`

Docker Compose permet de :

- Décrire toute l'application dans un fichier YAML
- Lancer tout avec une seule commande

Infrastructure as Code

- Structure d'un docker-compose.yml
 - services:
 - service1:
 - image: ...
 - service2:
 - build: ...
- Structure générale :
 - services
 - volumes (optionnel)
 - networks (optionnel)
 - ...

Docker Compose

Exemple Simple

services:

backend:

build: ./backend

ports:

– "8000:8000"

frontend:

build: ./frontend

ports:

– "8080:80"

depends_on:

– backend

- Quand vous faites :
 - **docker compose up**
- Docker :
 - Build les images
 - Crée un réseau dédié
 - Connecte les services
 - Lance les conteneurs

Docker Compose

création explicite d'un réseau

services:

backend:

build: ./backend

ports:

– "8000:8000" # publication : VM:8000 → conteneur backend:8000

networks:

– films-net # connecte le service au réseau films-net

frontend:

build: ./frontend

ports:

– "8080:80" # publication : VM:8080 → conteneur frontend:80

depends_on:

– backend # ordre de démarrage (ne garantit pas "backend prêt")

networks:

– films-net # même réseau → communication par nom

networks: # Déclaration du réseau (créé automatiquement par Docker Compose)

films-net:

driver: bridge # réseau bridge "user-defined" (DNS interne)

Docker Compose

Variable d'Environnement

```
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - APP_ENV=production
```

```
frontend:
  build: ./frontend
  ports:
    - "8080:80"
  env_file:
    - .env
  depends_on:
    - backend
```

Ajouter un Volume

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: pass
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```
volumes:
  pgdata: # Volume nommé
  driver: local # Driver par défaut
  driver_opts:
    type: none
    o: bind
    device: ./postgres-data # Exemple bind mount local
(optionnel)
```

Docker Compose

Commandes Compose

Lancer :

- `docker compose up`

Lancer en arrière-plan :

- `docker compose up -d`

Rebuild :

- `docker compose up --build`

Arrêter :

- `docker compose down`

Supprimer volumes :

- `docker compose down -v`

Gestion des mots de passe

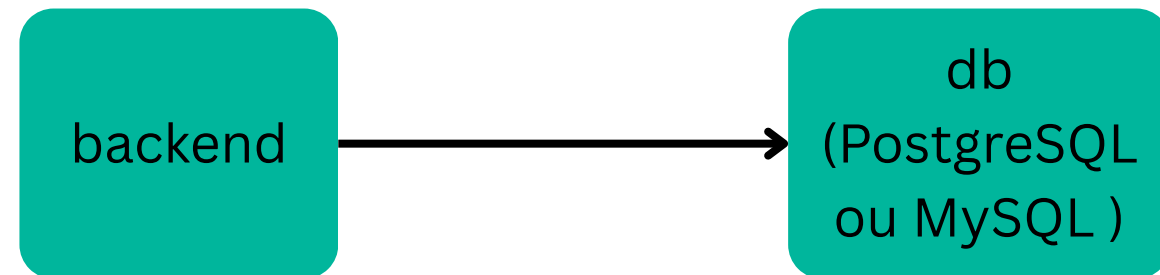
Le problème des mots de passe dans Docker

ENV DB_PASSWORD=123456

- Mauvaise pratique
- Visible dans l'image
- Visible avec docker inspect
- Visible dans l'historique Git

Gestion des mots de passe

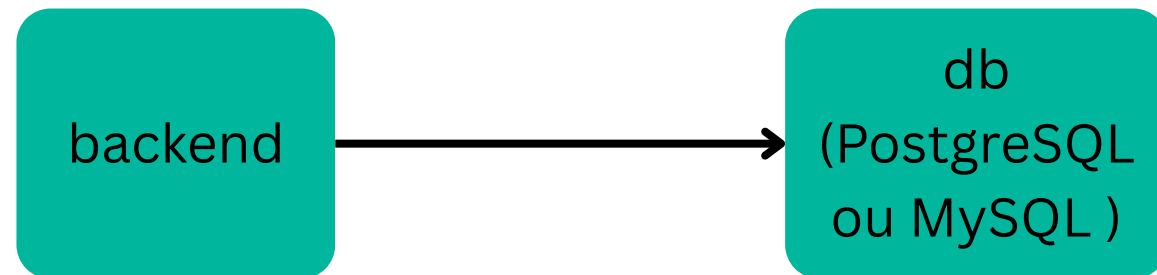
On a une application (backend) + une base PostgreSQL.



- Variables attendues par le backend
 - DB_HOST, DB_USER, DB_PASSWORD, DB_NAME

Gestion des mots de passe

On a une application (backend) + une base PostgreSQL.



- Variables attendues par le backend
 - DB_HOST, DB_USER, DB_PASSWORD, DB_NAME

services:

db:

image: postgres:16

environment:

POSTGRES_DB: appdb

POSTGRES_USER: appuser

TODO: mot de passe ?

backend:

build: ./backend

ports:

- "8000:8000"

environment:

DB_HOST: db

DB_USER: appuser

DB_NAME: appdb

TODO: DB_PASSWORD ?

depends_on:

- db

Gestion des mots de passe

On a une application (backend) + une base PostgreSQL.

- A) Mettre le mot de passe dans le Dockerfile (image)
 - **ENV DB_PASSWORD=supersecret**

services:

db:

image: postgres:16

environment:

POSTGRES_DB: appdb

POSTGRES_USER: appuser

TODO: mot de passe ?

backend:

build: ./backend

ports:

- "8000:8000"

environment:

DB_HOST: db

DB_USER: appuser

DB_NAME: appdb

TODO: DB_PASSWORD ?

depends_on:

- db

Gestion des mots de passe

On a une application (backend) + une base PostgreSQL.

- A) Mettre le mot de passe dans le Dockerfile (image)
 - **ENV DB_PASSWORD=supersecret**
- B) environment:
 - **DB_PASSWORD: supersecret**

services:

db:

image: postgres:16

environment:

POSTGRES_DB: appdb

POSTGRES_USER: appuser

TODO: mot de passe ?

backend:

build: ./backend

ports:

- "8000:8000"

environment:

DB_HOST: db

DB_USER: appuser

DB_NAME: appdb

TODO: DB_PASSWORD ?

depends_on:

- db

Gestion des mots de passe

On a une application (backend) + une base PostgreSQL.

- A) Mettre le mot de passe dans le Dockerfile (image)
 - **ENV DB_PASSWORD=supersecret**
- B) environment:
 - **DB_PASSWORD: supersecret**

C) Utiliser un fichier .env (ou env_file)

.env :

- **DB_PASSWORD=supersecret**

Compose :

- **env_file:**
 - **- .env**

services:

db:

image: postgres:16

environment:

POSTGRES_DB: appdb

POSTGRES_USER: appuser

TODO: mot de passe ?

backend:

build: ./backend

ports:

- "8000:8000"

environment:

DB_HOST: db

DB_USER: appuser

DB_NAME: appdb

TODO: DB_PASSWORD ?

depends_on:

- db

Gestion des mots de passe

On a une application (backend) + une base PostgreSQL.

- A) Mettre le mot de passe dans le Dockerfile (image)
 - **ENV DB_PASSWORD=supersecret**
- B) environment:
 - **DB_PASSWORD: supersecret**
- C) Utiliser un fichier .env (ou env_file)
.env :
 - **DB_PASSWORD=supersecret**Compose :
 - **env_file:**
 - - **.env**
- Recommandée (cours / TP) : C) .env + .gitignore
 - Le secret n'est pas dans l'image
 - Plus propre à maintenir que le YAML rempli de secrets
 - Facile à changer sans rebuild
- B) est acceptable en dev, mais...
 - Le secret est visible dans le compose
 - Peut être poussé sur Git par erreur
- A) est une mauvaise pratique
 - Le secret peut finir :
 - dans l'historique d'image
 - dans un registry
 - dans du code partagé
 - Très dur à “retirer” une fois publié

Gestion des mots de passe

```
.env :
POSTGRES_PASSWORD=supersecret
DB_PASSWORD=supersecret
docker-compose.yml :
services:
  db:
    image: postgres:16
    env_file:
      - .env
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```
backend:
  build: ./backend
  ports:
    - "8000:8000"
  env_file:
    - .env
  environment:
    DB_HOST: db
    DB_USER: appuser
    DB_NAME: appdb
    DB_PASSWORD: ${DB_PASSWORD}
  depends_on:
    - db
```

Ajouter dans .gitignore :
.env

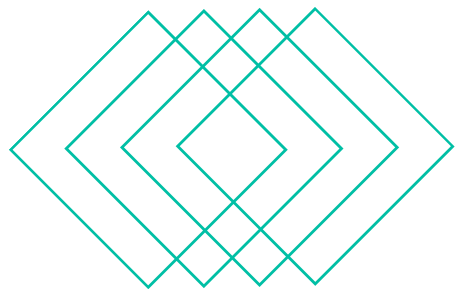


Gestion des mots de passe



En production, est-ce suffisant ?

- Est-ce que .env est sécurisé en production ?



Gestion des mots de passe

En production, est-ce suffisant ?

○ Est-ce que `.env` est sécurisé en production ?

- Visible avec `docker inspect`
- Visible dans variables d'environnement du conteneur
- Lisible par le process
- Pas chiffré par défaut

Même avec :

- `env_file`:
- `- .env`

On peut faire :

- **`docker inspect container`**

Les variables apparaissent.

Gestion des mots de passe

Docker Secrets

- Docker propose :
 - Docker Secrets
- Permet de :
 - Stocker les secrets de façon sécurisée
 - Ne pas les exposer comme variables
 - Les monter comme fichiers temporaires
 - Limiter l'accès au conteneur autorisé

Gestion des mots de passe

Docker Secrets

- Docker propose :
 - Docker Secrets
- Permet de :
 - Stocker les secrets de façon sécurisée
 - Ne pas les exposer comme variables
 - Les monter comme fichiers temporaires
 - Limiter l'accès au conteneur autorisé

Un secret :

- Est stocké par Docker
- N'est pas dans l'image
- N'est pas dans les variables d'environnement
- Est monté comme fichier dans :
 - **/run/secrets/**

Gestion des mots de passe

Créer un secret

Créer un fichier :

- db_password.txt
- Contenu : supersecret

Créer le secret :

- **docker secret create db_password db_password.txt**

Utiliser les secrets dans Compose (Swarm mode)

Docker secrets nécessite Swarm

Initialiser :

- **docker swarm init**

services:

db:

image: postgres:16

secrets:

- db_password

environment:

POSTGRES_PASSWORD_FILE: /run/secrets/db_password

secrets:

db_password:

external: true

Gestion des mots de passe

Ce qui se passe réellement

Dans le conteneur :

Le secret est disponible ici :

- **/run/secrets/db_password**

- Pas visible dans inspect
- Pas visible comme variable
- Fichier temporaire sécurisé

- Secret non stocké dans image
- Non exposé dans inspect
- Pas visible dans variables
- Gestion centralisée
- Limité aux services autorisés

Bonnes Pratiques & Pièges Fréquents

Ne jamais utiliser root

- Par défaut, les conteneurs tournent en root.
- Mauvaise pratique en production
- Dans Dockerfile :
 - RUN useradd -m appuser
 - USER appuser

Réduit les risques de sécurité

Ne pas mettre de secrets dans l'image

- `ENV DB_PASSWORD=123456`
- L'image peut être partagée
- Les secrets restent visibles
- Utiliser :
 - variables d'environnement
 - fichiers .env
 - Docker secrets

Bonnes Pratiques & Pièges Fréquents

Optimiser la taille des images

- python:3.11-slim
- node:alpine
- Supprimer fichiers inutiles
- Utiliser multi-stage build
- Utiliser .dockerignore

Bonnes Pratiques & Pièges Fréquents

Optimiser la taille des images

- python:3.11-slim
- node:alpine
- Supprimer fichiers inutiles
- Utiliser multi-stage build
- Utiliser .dockerignore

Toujours écouter sur 0.0.0.0

```
app.run(host="127.0.0.1")
```

- Dans Docker, il faut :
 - host="0.0.0.0"
- Sinon inaccessible depuis l'extérieur.

Bonnes Pratiques & Pièges Fréquents

Optimiser la taille des images

- python:3.11-slim
- node:alpine
- Supprimer fichiers inutiles
- Utiliser multi-stage build
- Utiliser .dockerignore

Toujours versionner les images

- myapp:latest
- Bon :
 - myapp:1.0
 - myapp:1.1
- En production :
 - éviter latest

Toujours écouter sur 0.0.0.0

```
app.run(host="127.0.0.1")
```

- Dans Docker, il faut :
 - host="0.0.0.0"
- Sinon inaccessible depuis l'extérieur.

Bonnes Pratiques & Pièges Fréquents

Optimiser la taille des images

- python:3.11-slim
- node:alpine
- Supprimer fichiers inutiles
- Utiliser multi-stage build
- Utiliser .dockerignore

Toujours versionner les images

- `myapp:latest`
- Bon :
 - `myapp:1.0`
 - `myapp:1.1`
- En production :
 - éviter latest

Toujours écouter sur 0.0.0.0

```
app.run(host="127.0.0.1")
```

- Dans Docker, il faut :
 - `host="0.0.0.0"`
- Sinon inaccessible depuis l'extérieur.

Pièges Fréquents

- Confondre EXPOSE et `-p`
- Utiliser localhost entre conteneurs
- Oublier les volumes pour DB
- Modifier code sans rebuild
- Lancer trop de conteneurs inutiles

Ce qu'il reste à apprendre

- **Docker Networking avancé**

- Subnets personnalisés
- Overlay networks
- Swarm networking

- **Docker Storage avancé**

- Drivers de stockage
- Volumes externes
- Sauvegarde & restauration

- **Docker Security**

- Rootless Docker
- Capabilities Linux
- Seccomp profiles
- Image scanning

- **Docker Registry avancé**

- Registry privé
- Docker Hub CI/CD
- Tagging strategy
- Digest vs tag

- **Docker Compose avancé**

- Profiles
- Healthcheck
- Multi-file compose
- Production vs dev config

- **Docker Swarm**

- Orchestration native Docker
- Services
- Scaling
- Load balancing

Problème : proxy de l'université

- À l'université, l'accès Internet passe souvent par un proxy.
 - docker pull échoue (timeout / connexion refusée)
 - apt-get, pip, npm échouent pendant le build (docker build)
 - Dans un conteneur : curl / ping / apt ne marchent pas

Exemples d'erreurs :

- i/o timeout
- proxyconnect tcp: dial tcp ...: connect: connection refused
- Temporary failure resolving ...
- Could not resolve host
- TLS handshake timeout
- Traduction : Docker n'a pas la bonne configuration réseau / proxy / DNS.

Problème : proxy de l'université

- Docker a plusieurs contextes réseau :
 - Machine (VM Ubuntu)
 - Docker daemon (qui fait pull)
 - Build (les RUN apt/pip/npm)
 - Conteneurs en exécution
- Si le proxy est configuré sur la VM mais pas sur Docker:
 - navigateur marche, apt sur terminal VM marche
 - **docker pull échoue**

Sur Ubuntu, le proxy peut être défini dans :

- variables : HTTP_PROXY, HTTPS_PROXY, NO_PROXY
- settings système
- fichiers de config entreprise

Commandes pour vérifier :

- **env | grep -i proxy**

config proxy pour le Docker daemon

- Docker pull est fait par dockerd, donc il faut configurer le proxy au niveau du service.
- Créer un override systemd :
 - `sudo mkdir -p /etc/systemd/system/docker.service.d`
 - `sudo nano /etc/systemd/system/docker.service.d/http-proxy.conf`
 -
- Mettre :
 - `[Service]`
 - `Environment="HTTP_PROXY=http://proxy:3128"`
 - `Environment="HTTPS_PROXY=http://proxy:3128"`
 - `Environment="NO_PROXY=localhost,127.0.0.1,.local"`

`sudo systemctl daemon-reload`
`sudo systemctl restart docker`

config proxy pour le Docker daemon

- Docker pull est fait par dockerd, donc il faut configurer le proxy au niveau du service.
- Créer un override systemd :
 - `sudo mkdir -p /etc/systemd/system/docker.service.d`
 - `sudo nano /etc/systemd/system/docker.service.d/http-proxy.conf`

- Mettre :
 - [Service]
 - `Environment="HTTP_PROXY=http://proxy:3128"`
 - `Environment="HTTPS_PROXY=http://proxy:3128"`
 - `Environment="NO_PROXY=localhost,127.0.0.1,local"`

Vérifier que Docker voit le proxy

- **`systemctl show --property=Environment docker`**
- **`docker info | grep -i proxy`**

Tester :

- **`docker pull hello-world`**
 - Si ça marche → le daemon est ok.

`sudo systemctl daemon-reload`
`sudo systemctl restart docker`

Proxy dans les builds

- Même si docker pull marche, les RUN apt/pip/npm peuvent échouer.
- On passe le proxy comme build-arg :

```
ARG HTTP_PROXY  
ARG HTTPS_PROXY  
ARG NO_PROXY
```

```
ENV HTTP_PROXY=$HTTP_PROXY  
ENV HTTPS_PROXY=$HTTPS_PROXY  
ENV NO_PROXY=$NO_PROXY
```

```
RUN apt-get update && apt-get install -y curl
```

```
docker build --build-arg HTTP_PROXY=http://proxy:3128 --build-arg  
HTTPS_PROXY=http://proxy:3128 --build-arg NO_PROXY=localhost,127.0.0.1 -t myimage .
```

Docker Compose (build + run)

- Compose permet : de passer des args au build et de passer des environment au runtime

services:

backend:

build:

context: ./backend

args:

HTTP_PROXY: http://proxy:3128

HTTPS_PROXY: http://proxy:3128

NO_PROXY: localhost,127.0.0.1,db

environment:

HTTP_PROXY: http://proxy:3128

HTTPS_PROXY: http://proxy:3128

NO_PROXY: localhost,127.0.0.1,db

Checklist de dépannage (rapide)

- La VM a internet ?
 - **curl -I https://google.com**
- Docker daemon a le proxy ?
 - **docker info | grep -i proxy**
- docker pull marche ?
 - **docker pull hello-world**
- Build marche ?
 - **docker build ...**
- Dans conteneur, curl marche ?
 - **docker exec -it <c> sh**
 - **curl -I https://example.com**